Behavioral Synthesis of ICs

Control Units Synthesis

Credentials: majority of information are from the following sources:

1. Barkalov, Alexander, Larysa Titarenko, Kamil Mielcarek, and Sławomir Chmielewski. *Logic Synthesis for FPGA-Based Control Units*. Springer International Publishing, 2020.
2. Barkalov, Alexander, Larysa Titarenko, and Jacek Bieganowski. "*Logic synthesis for finite state Machines Based on Linear Chains of States*." Studies in Systems, Decision and Control, Springer, Berlin (2018).

Carleton University

Outline

- **Introduction**
- **FSM-based Control Unit Design**
- **Microinstruction-based Control Unit Design**

Objectives

→ Understanding FSM-based control unit design

→ Understanding microinstruction-based control unit design

Control Unit Model

- The Control Unit orchestrates the execution of synthesized instructions, translating high-level programming commands into specific signals that manage the operation of hardware components.
- It ensures that each part of the system acts in sync and at the right moment, pivotal for efficiently executing complex algorithms on digital hardware.

Control Unit Model

→ As a rule, any digital system may be represented as a composition of a data-path and a control unit.

→ Such a representation is based on the principle of microprogram control proposed by M. Wilkes in 1951.

→ In accordance with this principle, any complex operation is represented as a sequence of elementary operations (microoperations).

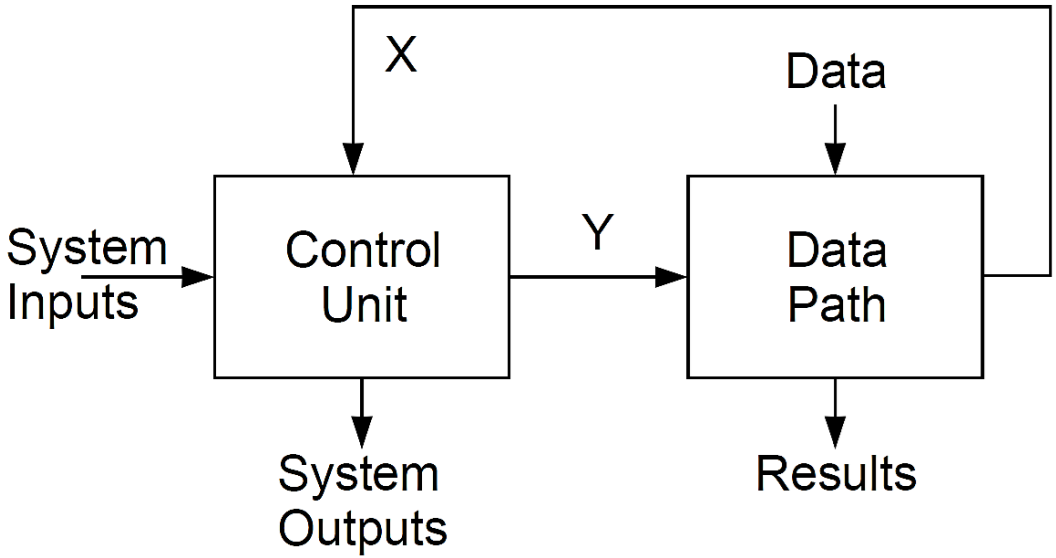
→ To control the order of execution of microoperations, special status signals (logical conditions) are used.

→ So, any complex operation is represented as a microprogram represented in terms of microoperations and logical conditions.

→ The microprogram is a particular form of a digital system's specification. Using this principle, it is possible to represent a digital system as it is shown in Figure1

Control Unit Model

→ **Figure1**. Model of digital system based on principle of microprogram control.



Control Unit Model

→ The control unit generates microoperations $y_n \in Y$, where $Y = \{y_1, \dots, y_N\}$ is a set of microoperations, and systems outputs.

→ Each microoperation (MO) evokes execution of some elementary operation by the data-path. The data-path executes operations under data and produces results of these operations.

→ Each cycle of operation is connected with producing logical conditions $x_e \in X$, where $X = \{x_1, \dots, x_L\}$ is a set of logical conditions. Values of logical conditions show the state of an operation's execution.

→ To generate a distributed in time sequence of microoperations, a control unit (CU) analyses values of logical conditions, as well as system inputs generated by environment of a system.

→ To communicate with environment, a system generates special system outputs. So, a control unit can be viewed as a brain of a digital system.

Control Unit Model

→ There are two main approaches in organization of the microprogram control.

- A microprogram could be implemented as a program in some high-level programming language. This way is used in microcontrollers.
  - It is the most universal approach for implementing control, for example, in embedded systems. But a rather low performance of such control units is a reverse of the medal (universality).
- The second way is a hardware implementation of control algorithms.
  - A microprogram could be kept into a special microprogram memory. This approach leads to microprogram control units.

→ Sometimes, these CUs are named automata with “programmed logic”.

Control Unit Model

→ A microprogram control unit (MCU) can be implemented using three main blocks:

- A sequencer (SQ),
- A register of microinstruction address (RAMI)
- A control memory (CM).

→ Its structural diagram is shown in Figure 2.

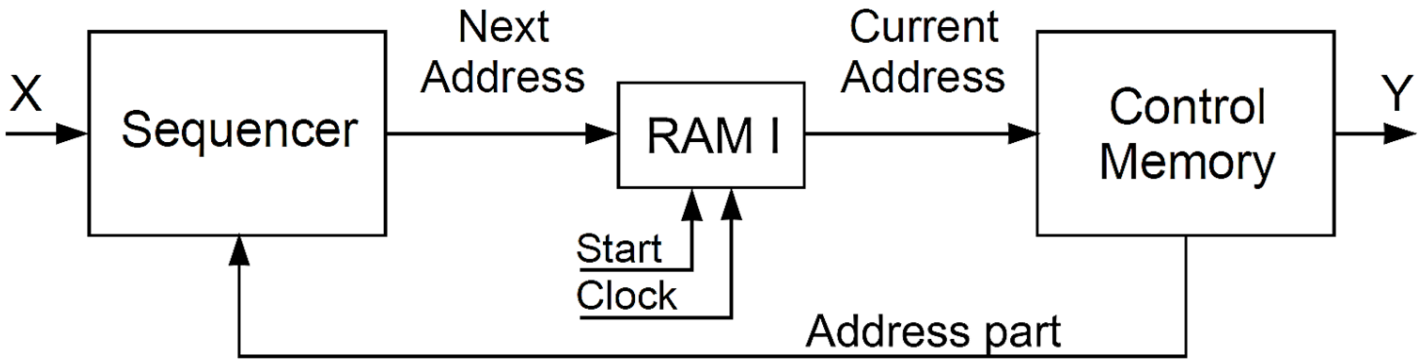
→ The SQ forms an address of transition to point the next microinstruction to be executed.

→ The RAMI is controlled by pulses Start and Clock. The pulse Start zeroes the RAMI; it corresponds to the beginning of operation. The pulse Clock shows the cycles of MCU. It permits changing content of RAMI.

→ A microprogram is represented as a list of microinstructions kept into the CM.

Control Unit Model

→ **Figure 2**. Structural diagram of MCU



Control Unit Model

→ Each microinstruction includes an operation part containing information about microoperations to be executed and address part having information about the next address.

→ The SQ analyses a logical condition x_e pointed in the address part. Basing on this analysis, the next address is generated.

→ For **example**, there is a special multiplexer (MX) in the SQ. Its informational inputs are connected with logical conditions $x_e \in X$ and its control inputs are connected with bits representing a code $K(x_e)$ (Figure 3a).

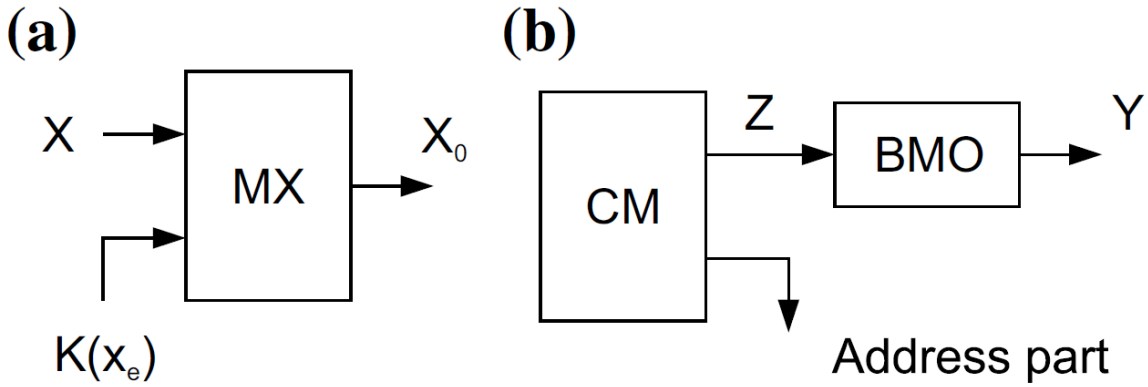
→ The code $K(x_e)$ is kept into the address part. It points which logic condition should be analyzed.

→ A variable x_e is equal to the value of a logical condition represented by $K(x_e)$.

→ This approach has turned into the method of replacement of logical conditions used for optimization of modern CUs.

Control Unit Model

→ **Figure 3**. Organization of MCU blocks



Control Unit Model

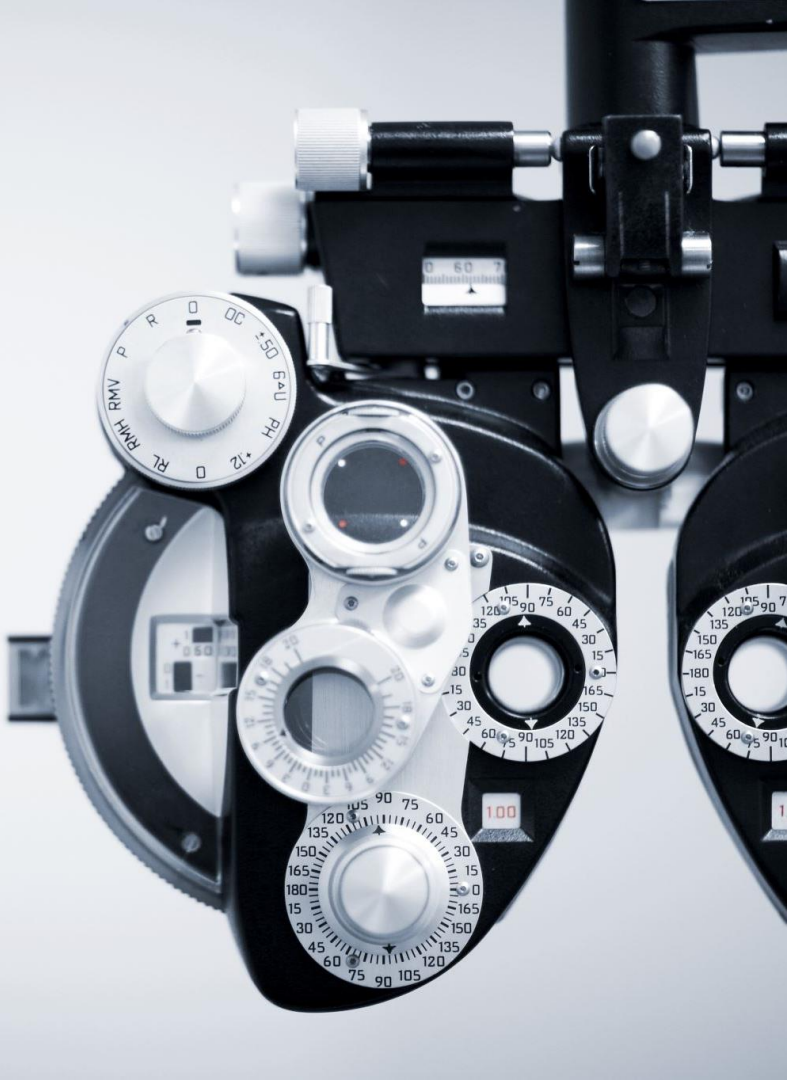
→ In the fifties, the volume of the control memory was extremely limited. This has necessitated reducing the length of the operation part of microinstruction.

→ To do it, the collections of microoperations (CMO) were encoded using additional variables from the set Z.

→ Then, these codes were decoded by a block of microoperations (BMO) into microoperations $y_n \in Y$.

→ It leads to a two-level circuit used for implementing the microoperations (see Figure 3b).

→ This method is used now by the designers of modern control units.



Finite State Machine Design

Control Unit Model: FSM

→ The second way of implementing CUs is connected with finite state machines (FSM).

→ An FSM is a sequential circuit. Its outputs depends on pre-history of operation.

→ The pre-history is represented by internal states forming the set $A = \{a_1, \dots, a_M\}$.

→ To implement an FSM logic circuit, the states are encoded by binary codes $K(a_m)$ having R bits.

→ To encode the states, internal state variables are used forming the set $T = \{T_1, \dots, T_R\}$.

→ These codes are kept into a special register (RG).

→ As a rule, an RG consists of flip-flops having informational inputs of type D.

→ To change the content of RG, special input memory functions $D_r \in \Phi$ are used where $\Phi = \{D_1, \dots, D_R\}$.

Control Unit Model: FSM

→ An FSM could be implemented as a composition of combinational circuit (CC) and RG (Figure 4).

→ The pulses **Start** and **Clock** have the same meaning as for MCU. So, the CC is represented by the following systems of Boolean functions (SBF):

$$\Phi = \Phi(T, X);$$

$$Y = Y(T, X).$$

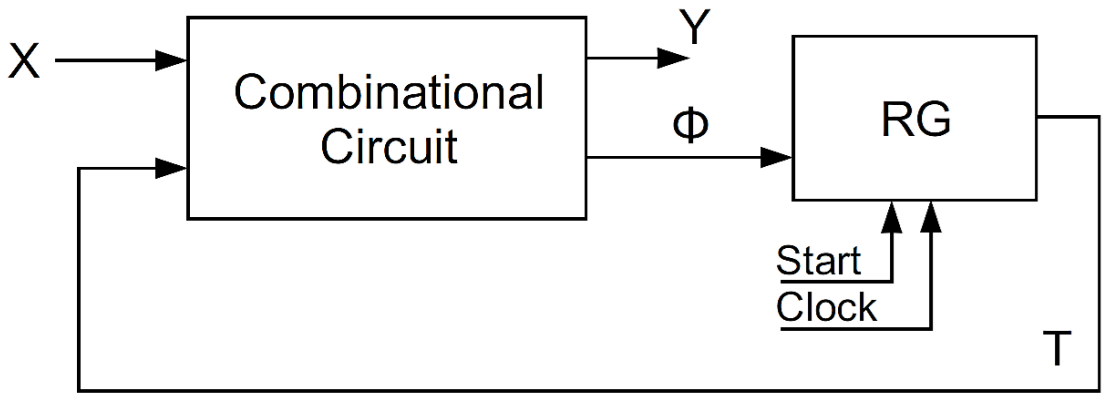
→ This system corresponds to a Mealy FSM.

→ In the case of Moore FSM, output functions are represented by the following system:

$$Y = Y(T).$$

Control Unit Model: FSM

→ **Figure 4**. Structural diagram of FSM



Control Unit Model: FSM

- In the Mealy FSMs outputs $y_n \in Y$ depend on both logical conditions $x_e \in X$ and state variables $T_r \in T$.
- But in the Moore FSM outputs depend only on state variables.

→ This difference leads to different methods of hardware reduction for their logical circuits.

→ There are many ways for specifying a behavior of an FSM. It can be specified by:

1) state transition graph (STG);
2) state transition table;
3) program in a hardware description language such as Verilog or VHDL.
4) graph-schemes of algorithm (GSA)

Control Unit Model: FSM

→ An Algorithmic representation includes four types of vertices (Figure 5).

→ Each algorithm includes exactly a single initial vertex corresponding to the starting point of a control algorithm (Figure 5a) and a single final vertex (Figure 5b).

→ An algorithm contains a finite amount of operational and conditional vertices.

→ Each operational vertex (Figure 5c) contains an CMO $Y_q \subseteq Y$.

→ It includes microoperations executed in the same instant.

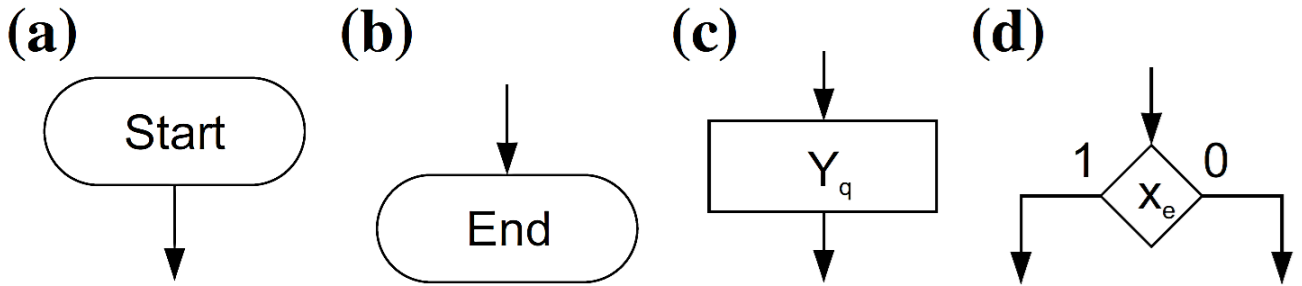
→ It has a single input and a single output.

→ A conditional vertex (Figure 5d) contains some logical condition $x_e \in X$. It has two outputs marked by the logic values of this condition $x_e \in X$.

→ The conditional vertices are used to organize the branching in the control algorithm.

Control Unit Model: FSM

→ **Figure 5**: Types of vertices in an algorithmic representation.



Control Unit Model: FSM

→ **Example**: Figure 6 shows an algorithm and its corresponding graph-schemes of algorithm.

→ For an algorithm with M states, $A = \{a_1, \dots, a_M\}$, at least R state variable are required to encode the states, using a binary coding system:

$$R = \lceil log_2^M \rceil$$

→ For the algorithm of the figure 6, $R = \lceil log_2^5 \rceil = 3$.

→ It gives the sets $T = \{T_1, T_2, T_3\}$ and $\Phi = \{D_1, D_2, D_3\}$. Let us encode the states $a_m \in A$ in the trivial way: $K(a_1) = 000, \dots, K(a_5) = 100$.

→ The DST of Moore FSM is constructed on the base of STG. The STG vertices correspond to FSM states, whereas arks to transitions between the states $a_m \in A$.

→ If a CMO $Yq \subseteq Y$ is generated in the state $a_m \in A$, then it is written near the corresponding vertex of STG. The arks are marked by input signals causing transitions $\langle a_m, a_s \rangle$.

→ Input signals are conjunctions of logical conditions corresponding to a path in GSA Γ leading from a_m into a_s. In the discussed case, the STG has $M = 5$ vertices and $H = 7$ arks (Figure 6b). Each ark corresponds to a single row of direct structure table (DST).

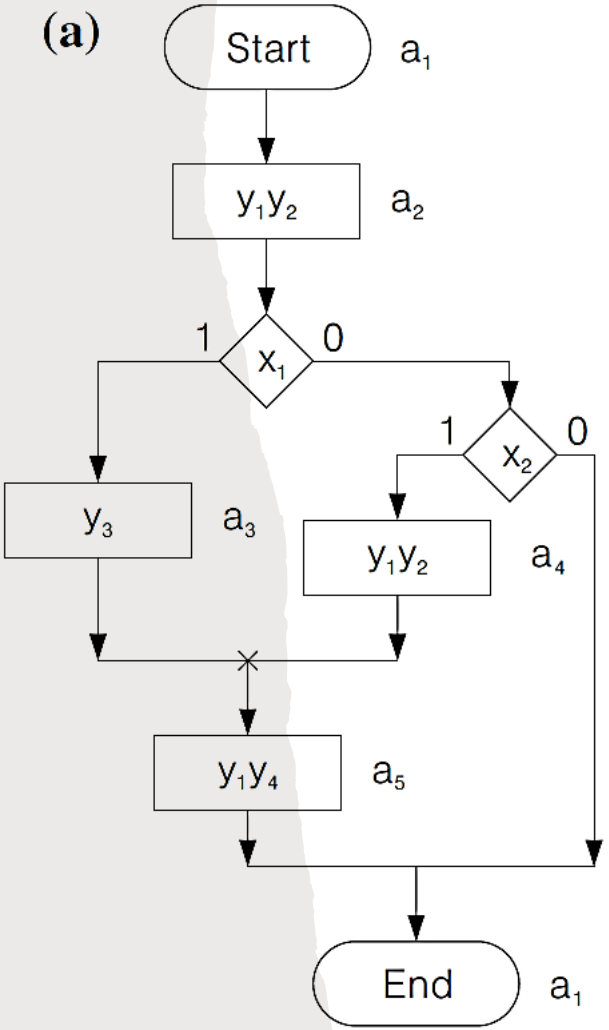
Control Unit Model: FSM

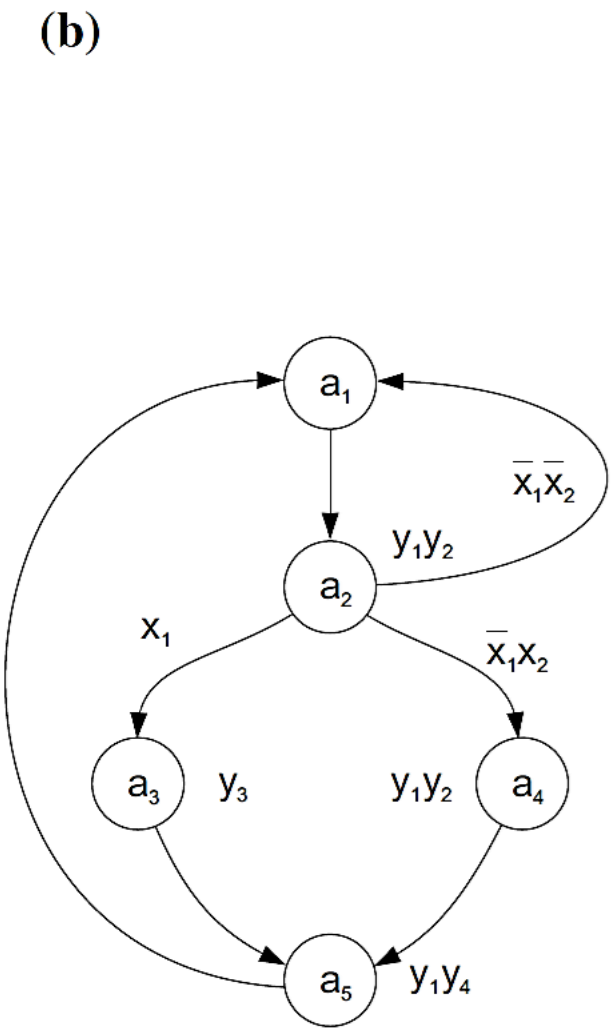
→ **Figure 6**:

a) A sample algorithm.

b) Corresponding graph-schemes of algorithm .

c) Sequence of operations





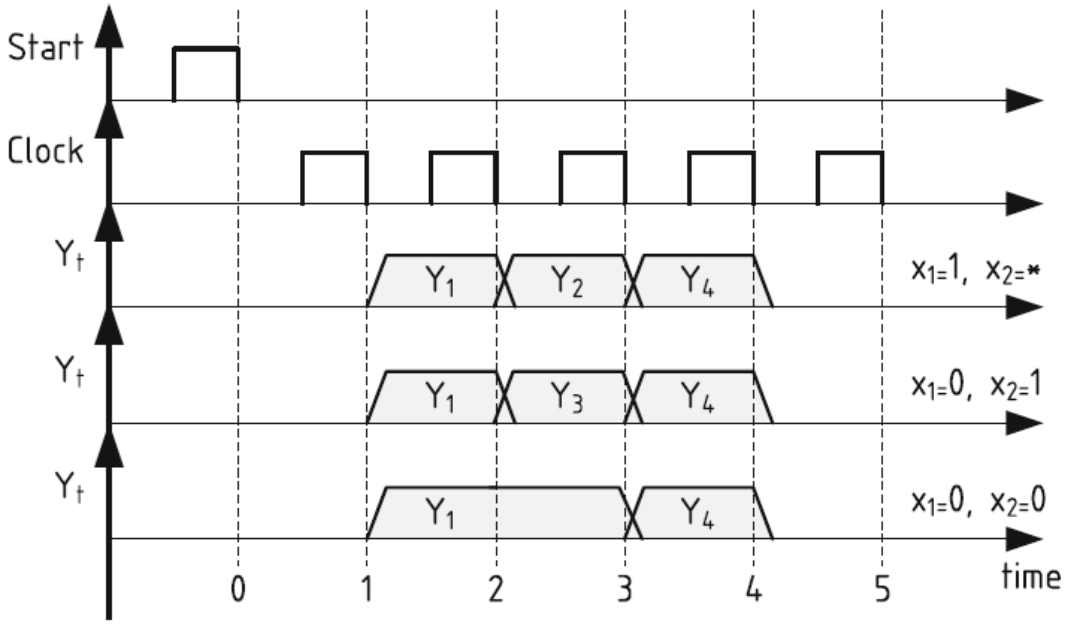
Control Unit Model: FSM

→ **Figure 6**. Sequence of the operation in the sample control unit.

a) A sample algorithm.

b) Corresponding graph-schemes of algorithm .

a) Sequence of operations



Control Unit Model: FSM

→ Table 1: Direct structure table of Moore FSM for Figure 6.

- a_m: current state,
- a_s: next state,
- $K(am)$: state's code,
- X_h: condition.

a_m	$K(a_m)$	a_s	$K(a_s)$	X_h
a_1	000	a_2	001	1
$a_2(y_1 y_2)$	001	a_3	010	x_1
		a_4	011	$\bar{x_1} x_2$
		a_1	000	$\bar{x_1} \bar{x_2}$
$a_3(y_3)$	010	a_5	100	1
$a_4(y_1 y_2)$	011	a_5	100	1
$a_5(y_1 y_4)$	100	a_1	000	1

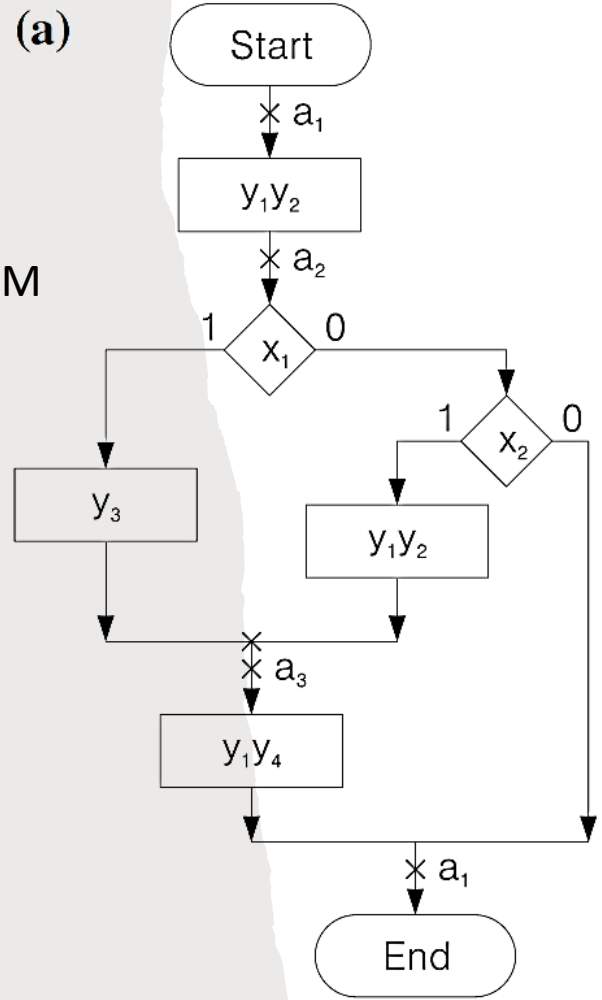
Note: For details of FSM design steps refer to Appendix A in this presentation slides.

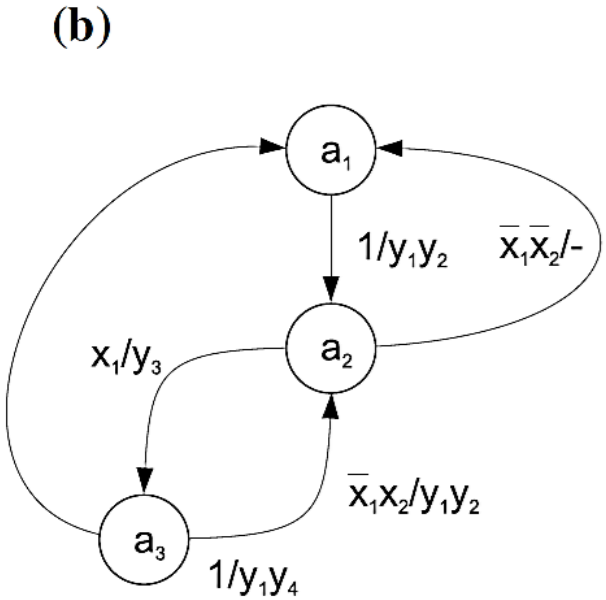
Control Unit Model: FSM

Figure 8. A sample Mealy FSM

(a) Marked GSA

(b) STG,





Control Unit Model: FSM

→ Table 2: Direct structure table of Moore FSM for Figure 8.

- a_m: current state,
- a_s: next state,
- $K(am)$: state's code,
- X_h: condition,
- Y_h: output.

a_m	$K(a_m)$	a_s	$K(a_s)$	X_h	Y_h
a_1	00	a_2	01	1	$y_1 y_2$
a_2	01	a_3	10	x_1	y_3
		a_3	10	$\bar{x_1} x_2$	$y_1 y_2$
		a_1	00	$\bar{x_1} \bar{x_2}$	–
a_3	10	a_1	00	1	$y_1 y_4$

Note: For details of FSM design steps refer to Appendix A in this presentation slides.

Control Unit Model: FSM

→ In a Mealy FSM, the outputs are not necessarily synchronized with clock. Therefore, Mealy FSM logic circuit should include additional flip-flops to be stable.

→ To stabilize the Mealy FSM operation, it is possible to use either register RY (Figure 9a) or RX (Figure 9b).

→ In both cases, any fluctuations of logical conditions do not change microoperations $y_n \in Y$.

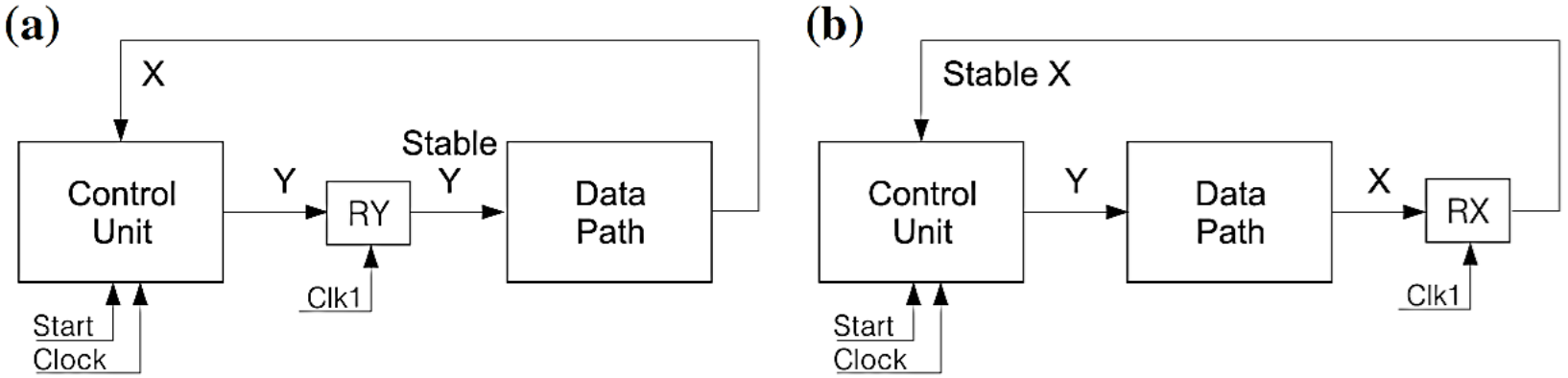


Figure 9. Stabilization of Mealy FSM using (a) RY or (b) RX.

Control Unit Model: FSM

→ It is possible that a control unit generates two kinds of control signals. Some signals exist only a short part of a CU cycle. They can be treated as the outputs of Mealy FSM.

→ Other signals exist during the whole cycle. These signals have types of Moore FSM outputs. Such control units could be represented by a model of combined finite state machine.

→ From the theoretical point of view, a CFSM could be represented by the following vector:

$$S = \langle A, X, Y^1, Y^2, \delta, \lambda_1, \lambda_2, a_1 \rangle.$$

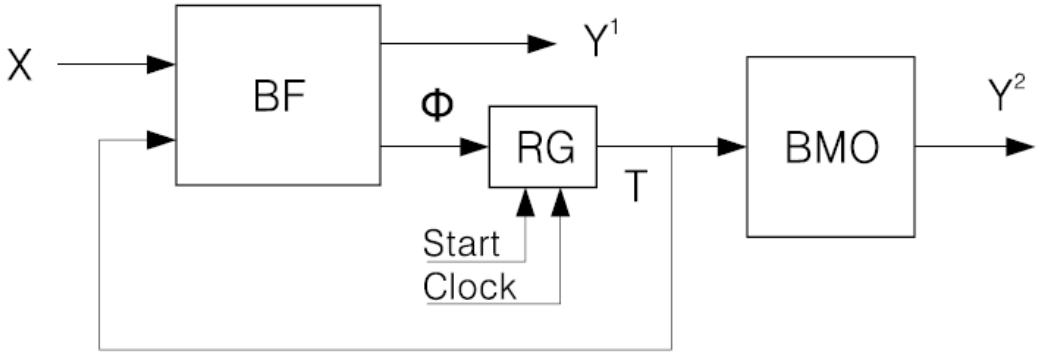
→ where, the set A includes M internal states, the set X consists of L logical conductions.

→ The set Y^1 includes N_1 microoperations of Mealy FSM. The set Y^2 includes N_2 microoperations of Moore FSM. The set Y includes $N = N_1 + N_2$ elements.

→ Note that, here, the following relations are true: $Y^1 \cap Y^2 = \emptyset$; $Y^1 \cup Y^2 = Y$.

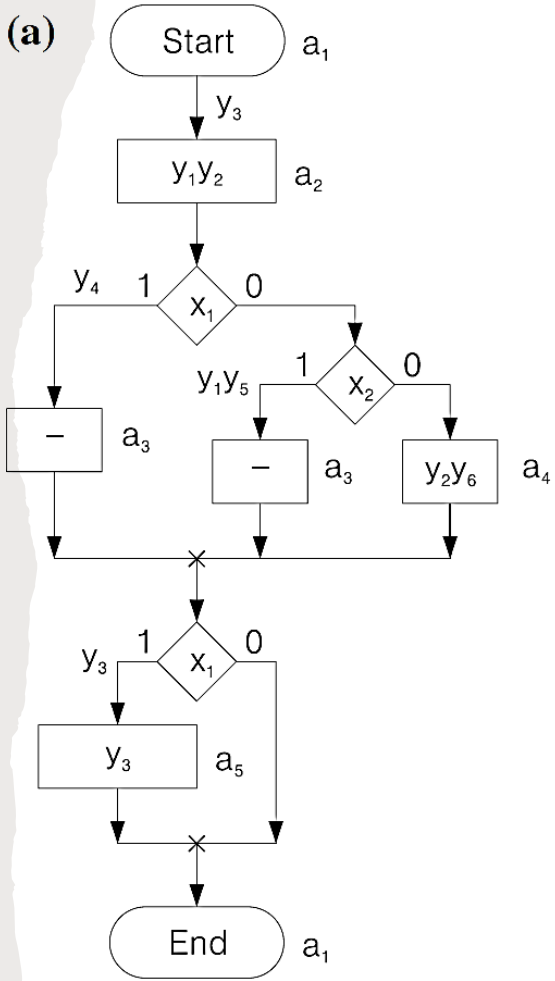
Control Unit Model: FSM

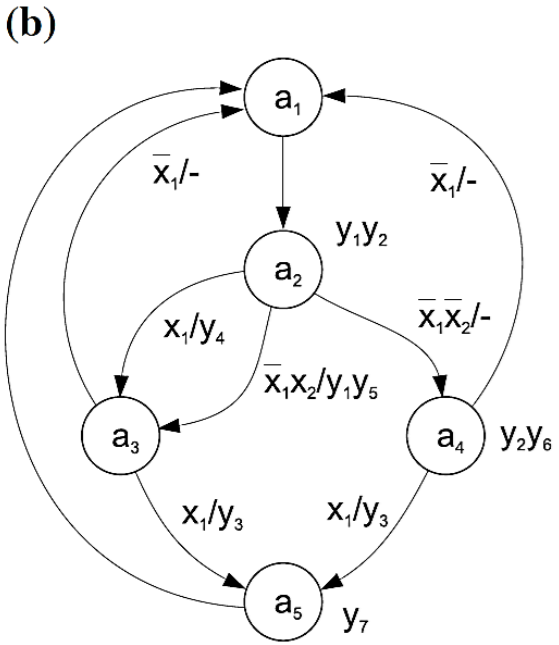
→ **Figure 10**. Structural diagram of combined finite state machine



Control Unit Model: FSM

- **Figure 11**. A sample combined finite state machine
  a) Algorithmic representation
  b) State transition graph





Control Unit Model: FSM

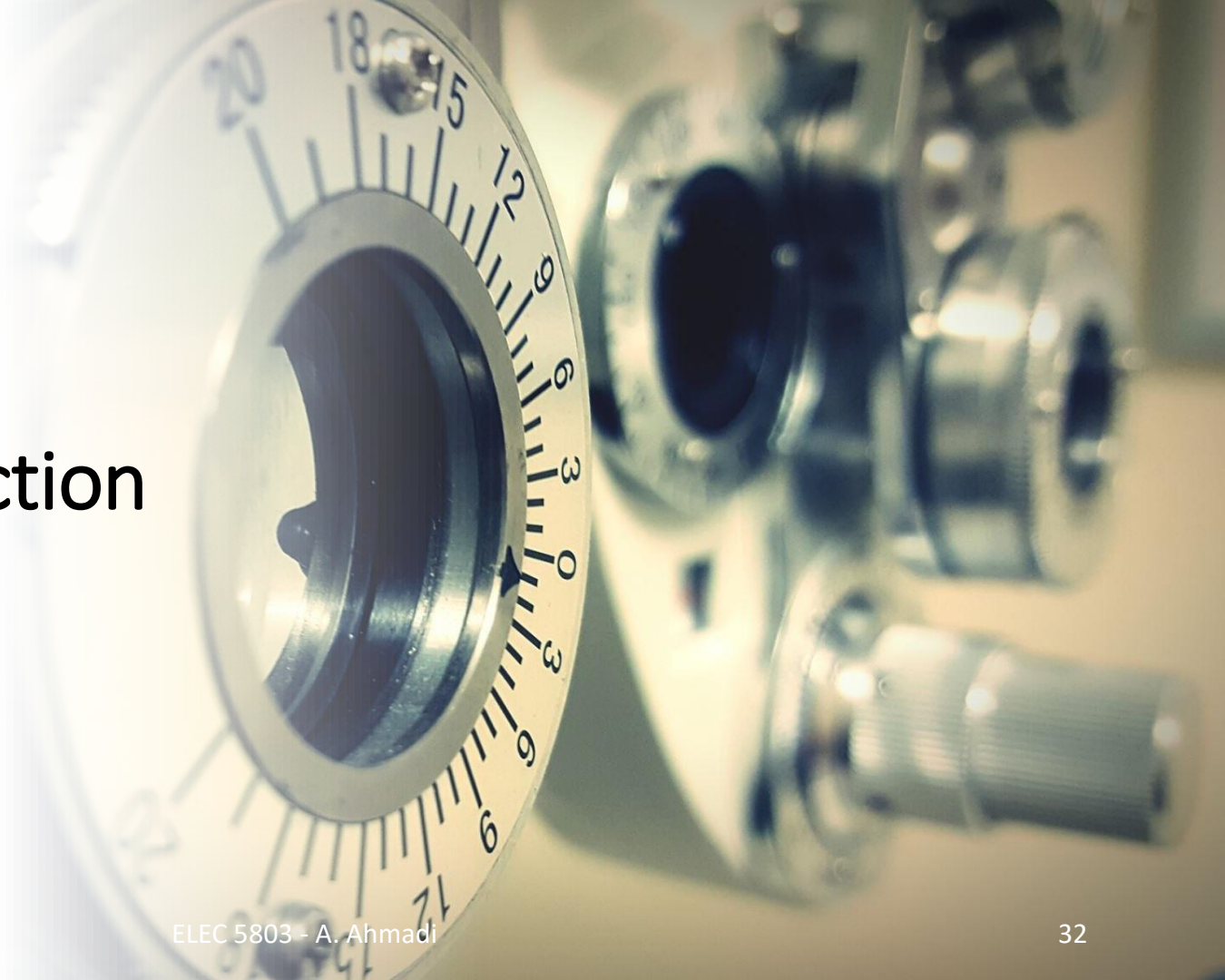
→ Table 2: Direct structure table of Moore FSM for Figure 11.

- a_m: current state,
- a_s: next state,
- $K(am)$: state's code,
- X_h: condition,
- Y_h: output.

a_m	$K(a_m)$	a_s	$K(a_s)$	X_h	Y_h^1
a_1	000	a_2	100	1	y_3
$a_2(y_1y_2)$	100	a_3	010	x_1	y_4
		a_3	010	$\bar{x_1}x_2$	y_3y_5
		a_4	101	$\bar{x_1}\bar{x_2}$	–
$a_3(-)$	010	a_5	001	x_1	y_3
		a_1	000	$\bar{x_1}$	–
$a_4(y_2y_6)$	101	a_5	001	x_1	y_3
		a_1	000	$\bar{x_1}$	–
$a_5(y_7)$	001	a_1	000	1	–

Note: For details of FSM design steps refer to Appendix A in this presentation slides.

Microinstruction Control Unit



Control Unit Model: Microprogram

→ It is necessary to take into account the nature of a control algorithm to optimize characteristics of a control unit.

→ If the set of vertices for a given algorithm includes more than 75% of operator vertices, it means that a corresponding FSM possesses a lot of unconditional transitions. This property can be used for simplifying the system of input memory functions.

→ As a rule of thump, the simplification of input memory functions is possible if the state register is replaced by some more complex device.

→ Two approaches are possible.

- In the first case a shift register replaces the state register. This approach requires sophisticated algorithms for the state assignment. Because of it, shift register did not find a wide application in FSMs.
- The second approach is connected with replacement of the state register by a state counter. This approach has found a wide application in the case of microprogram control units

Control Unit Model: Microprogram

→ Microprogram control units are based on the operational - address principle for presentation of control words (microinstructions) kept in a special control memory.

→ The typical method of MCU design includes the following steps:

1. Transformation of initial graph-scheme of algorithm.
2. Generation of microinstructions with given format.
3. Microinstruction addressing.
4. Encoding of operational and address parts of microinstructions.
5. Construction of control memory content.
6. Synthesis of logic circuit of MCU using given logical elements.

Control Unit Model: Microprogram

→ The mode of microinstruction addressing affects tremendously the method of MCU synthesis.

→ Three particular addressing modes are known:

1. Compulsory addressing of microinstructions.
2. Natural addressing of microinstructions.
3. Combined addressing of microinstructions.

→ In the first case, the address of microinstruction is kept into a register. This address be viewed as a state code, whereas a microinstruction can be viewed as an FSM state.

→ In the last two cases, the address of microinstruction is kept into a counter.

Control Unit Model: Microprogram

→ In the common case, microinstruction formats include the following fields: FY, FX, FA0 and FA1.

- The field FY, operational part of the microinstruction, contains information about microoperations $y_n \in Y$ ($t = 0, 1, \ldots$), which are executed in cycle t of control unit operation.
- The field FX contains information about logical condition $x_l^t \in X$, which is checked at time t
- The filed FA0 contains next microinstruction address A_{t+1} (transition address), either in case of unconditional transition, or if $x_l^t = 0$.
- The field FA1 contains next microinstruction address for the case when $x_l^t = 1$.

→ The fields FX, FA0 and FA1 form the address part of microinstruction.

Control Unit Model: Microprogram

→ There are two microinstruction formats in case of natural microinstruction addressing (see Figure 12):

→ operational microinstructions corresponding to operator vertices of GSA

→ control microinstructions corresponding to conditional vertices of GSA.

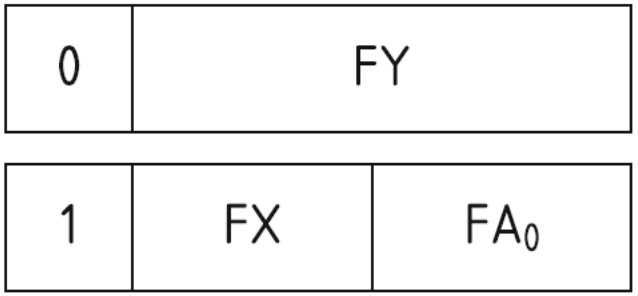


Figure 12: Microinstruction formats for MCU with natural addressing of microinstructions

Control Unit Model: Microprogram

→ First bit of each format represents field FA, used to recognize the type of microinstruction.

→ Let FA =0 correspond to operational microinstruction and FA =1 to control microinstruction.

→ As follows from Figure 11, next address is not included in operational microinstructions.

→ The same is true for the case, when a logical condition to be checked is equal to 1.

→ In both cases mentioned above current address A_t is used to calculate next address:

$$A^{t+1} = A^t + 1.$$

→ Hence, the following rule is used for next address calculation:

$$A^{t+1} = \begin{cases} A^t + 1 \text{ if } [FA]^t = 0; \\ A^t + 1 \text{ if } (x_l^t = 1) \wedge ([FA]^t = 1); \\ [FA_0]^t \text{ if } (x_l^t = 0) \wedge ([FA]^t = 1); \\ [FA_0]^t \text{ if } ([FX]^t = \emptyset) \wedge ([FA]^t = 1). \end{cases}$$

Control Unit Model: Microprogram

→ Two first records in the previous equation show that a counter should be included into the structure of MCU with natural addressing.

→ This counter is named a counter of microinstruction address (CAMI). The structure diagram of MCU includes a block of addressing (CFA), a control memory (CM), a block of microoperations (BMO) and a flip-flop TF used for fetching microinstructions from CM (see Figure 13).

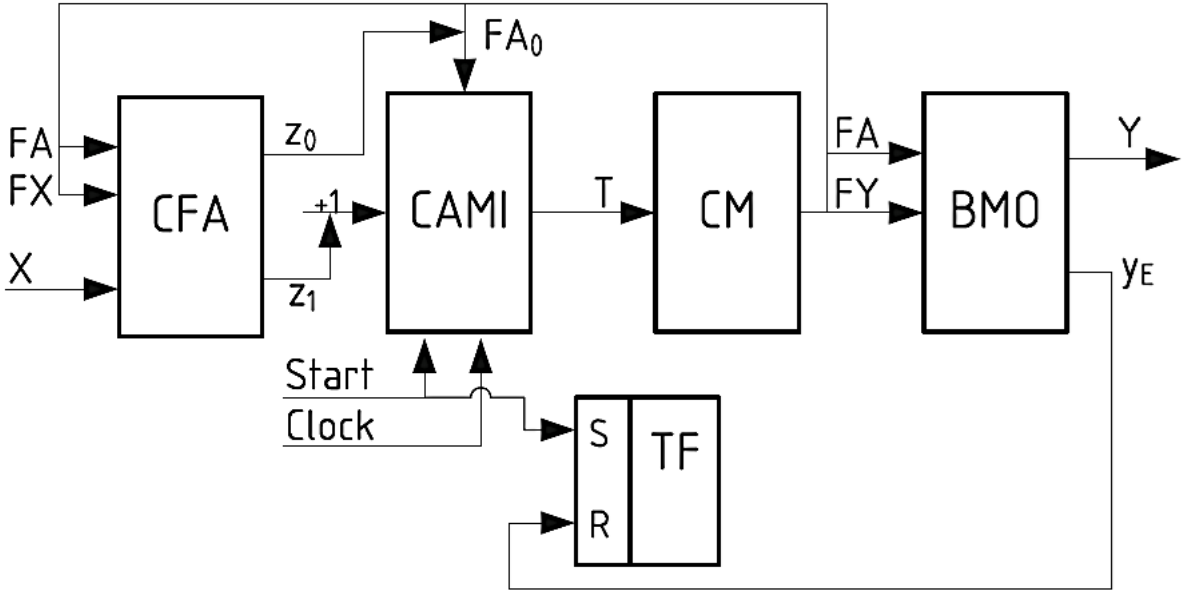
→ The MCU presented in Figure 13 has one serious drawback. Only a single logic condition is checked during one cycle of MCU's operation.

→ If an GSA includes a lot of multidirectional transitions, the performance of MCU is low. But it has very simple circuit of CFA, which can be implemented using just a multiplexer.

→ To improve the performance of MCU, the programmable logic arrays were used for implementing the circuit of CFA.

Control Unit Model: Microprogram

→ **Figure 13**: Structural diagram of MCU with natural addressing of microinstructions



Control Unit Model: Microprogram

→ This MCU of Figure 13 operates in the following manner.

- The pulse Start initiates loading of start address into CAMI. At the same time flip-flop TF is set up.
- Let an address A_t be located in CAMI at time t ($t = 0, 1, \ldots$).
- If this address determines an operational microinstruction, the block BMO generates microoperations $y_n \in Y$ and the sequencer CFA produces signal $z1$.
- If this address determines a control microinstruction, microoperations are not generated, and the sequencer produces either signal $z0$ (corresponding to an address loaded from the field FA0 or signal $z1$ (it corresponds to adding 1 to the content of CAMI).
- The content of counter CAMI can be changed by pulse Clock.
- If variable y_E is generated by BMO, then the flip-flop TF is cleared and operation of MCU terminated.

Control Unit Model: Microprogram

→ Due to their flexibility, the PLAs found applications in design of control units. They were used in FSMs to implement both functions Φ and

→ In the case of MCU, the block CFA can be implemented using a multiplexer, however, in literature it is proposed to be replaced by PLAs. This change allows execution of multidirectional transitions in a single cycle.

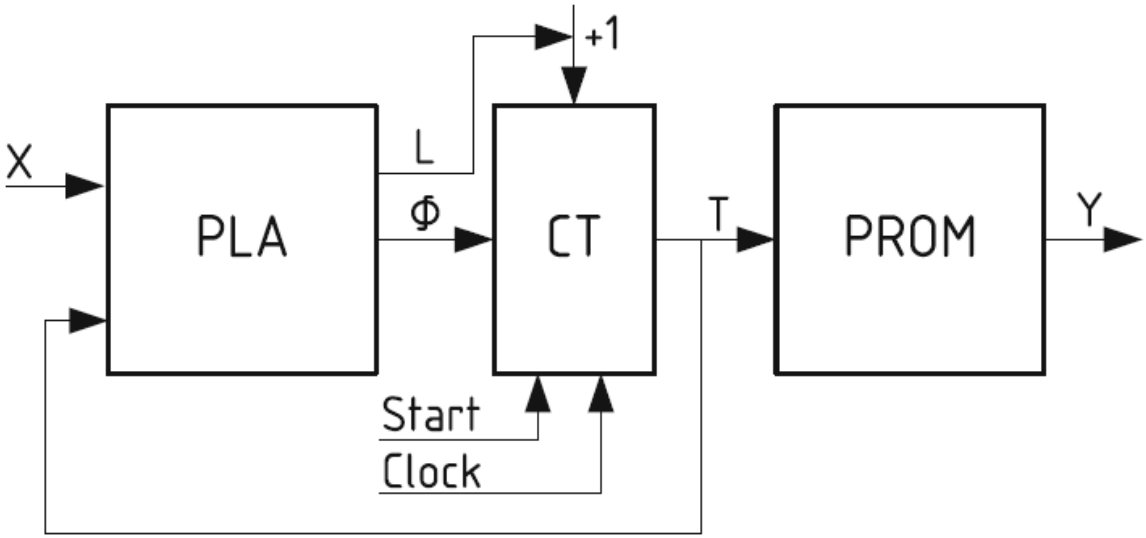
→ Having said that, the main drawback of this approach is the necessity of re-design of CFA if there are some changes in microprogram to be implemented. The structural diagram of PLA-based MCU is shown in Figure 14.

→ In this model, the control memory is implemented as PROM. The variable L is used for incrementing the counter CT.

→ If both blocks CFA and CM are implemented with PLAs, it leads to either Mealy or Moore FSM. The structural diagram of PLA-based Moore FSM is shown in figure 15.

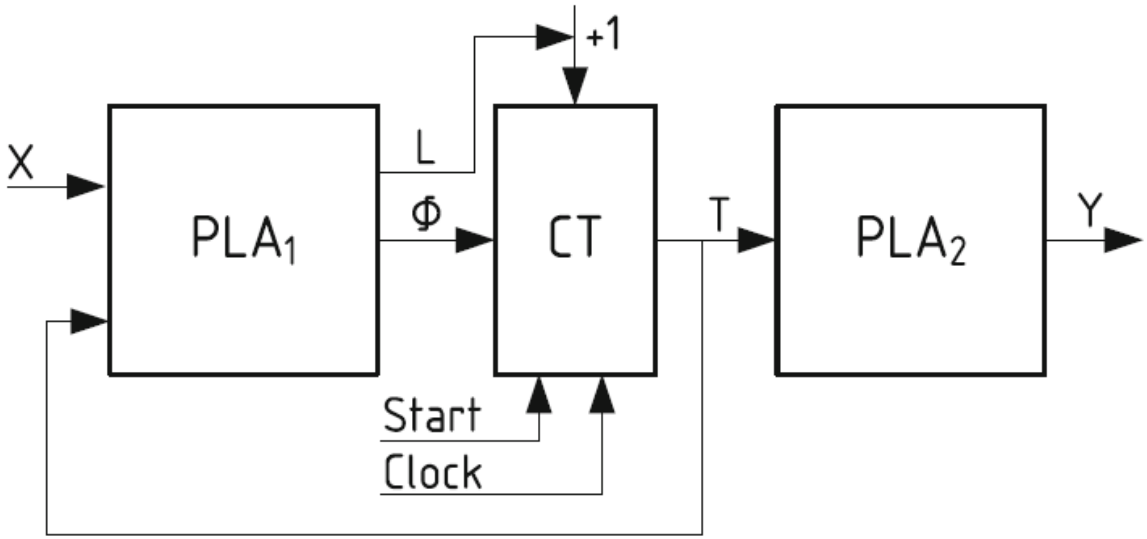
Control Unit Model: Microprogram

→ **Figure 14**: Structural diagram of PLA-based MCU



Control Unit Model: Microprogram

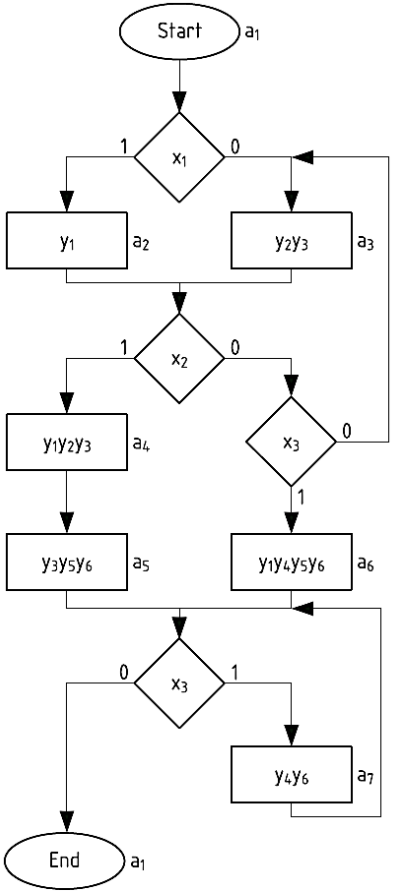
→ **Figure 15**: Structural diagram of PLA-based Moore FSM



Control Unit Model: Microprogram

→ A sample algorithm and table

B_i	$K(B_i)$	a_s	$K(a_s)$	X_h
B_1	000	a_2	001	x_1
		a_3	011	$\bar{x}_1$
B_2	0*1	a_4	010	x_2
		a_6	101	$\bar{x}_2 x_3$
		a_3	011	$\bar{x}_2 x_3$
B_3	010	a_5	100	1
B_4	1**	a_7	110	x_3
		a_1	000	$\bar{x}_3$

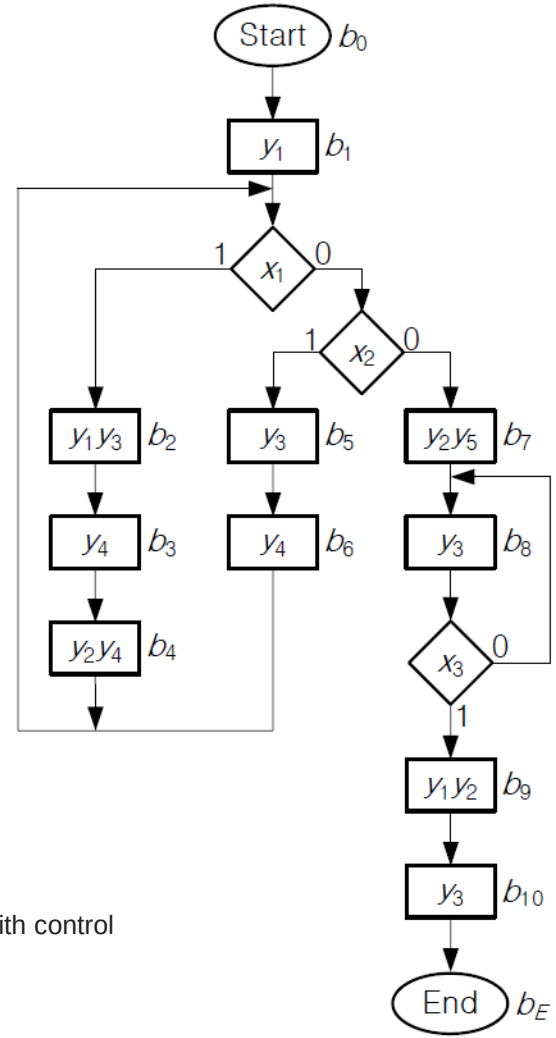


Control Unit Model: Microprogram

- Initial linear graph-scheme of algorithm

Address $T_1T_2T_3T_4$	y_0	FY						Remark	
		FB						b_q	B_i
			y_2	y_3	y_4	y_5	y_E		
0000	1	1	0	0	0	0	0	b_1	B_1
0001	0	0	*	*	*	*	*	O_1	B_1
0010	1	0	1	1	0	0	0	b_2	B_1
0011	1	0	0	0	1	0	0	b_2	B_1

B_i	$K(B_i)$	b_q	$A(b_q)$	X_h	Φ_h	h
B_1	0	b_2	0010	x_1	D_3	1
		b_5	0110	$\bar{x}_1x_2$	D_2D_3	2
		b_7	1001	$\bar{x}_1\bar{x}_2$	D_1D_4	3
B_2	1	b_8	1010	$\bar{x}_3$	D_1D_2	4
		b_9	1100	x_3	D_1D_3	5



Barkalov, Alexander, Larysa Titarenko, and Jacek Bieganowski. "Synthesis of microprogram control unit with control microinstructions." *IFAC Proceedings Volumes* 42, no. 21 (2009): 201-204.

Appendix A

Synchronous Sequential Logic Design

Credits: majority of information are from the textbook:

Digital Design with an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th Edition

Carleton University

Analysis of Clocked Sequential Circuits

→ The behavior of a clocked sequential circuit is determined from the inputs, the outputs, and the states (its flip-flops).

(*inputs*, *present states*) → (*outputs*, *next states*)

→ The outputs and the next state are both a function of the inputs and the present state.

→ The analysis of a sequential circuit consists of obtaining a table or a diagram for the time sequence of inputs, outputs, and internal states.

→ It is also possible to write Boolean expressions that describe the behavior of the sequential circuit.

→ These expressions must include the necessary time sequence, either directly or indirectly.

Analysis of Clocked Sequential Circuits

→ Analysis describes what a given circuit will do under certain operating conditions.

→ A logic diagram is recognized as a clocked sequential circuit if it includes flip-flops with clock inputs.

→ The flip-flops may be of any type, and the logic diagram may or may not include combinational logic gates.

→ For a systematic sequential circuits, we introduce an algebraic representation for specifying the next-state condition in terms of the present state and inputs.

→ A state table and state diagram are used to describe the behavior of the sequential circuit.

→ Another algebraic representation for specifying the logic diagram of sequential circuits is based on state equations.

→ The behavior of a clocked sequential circuit can be described algebraically by means of state equations.

State Equations

→ A state equation (also called a transition equation) specifies the next state as a function of the present state and inputs.

→ Consider the sequential circuit shown in next slide. This circuit acts as a 0-detector by asserting its output when a 0 is detected in a stream of 1's.

→ Circuit consists of two D flip-flops A and B, an input x and an output y.

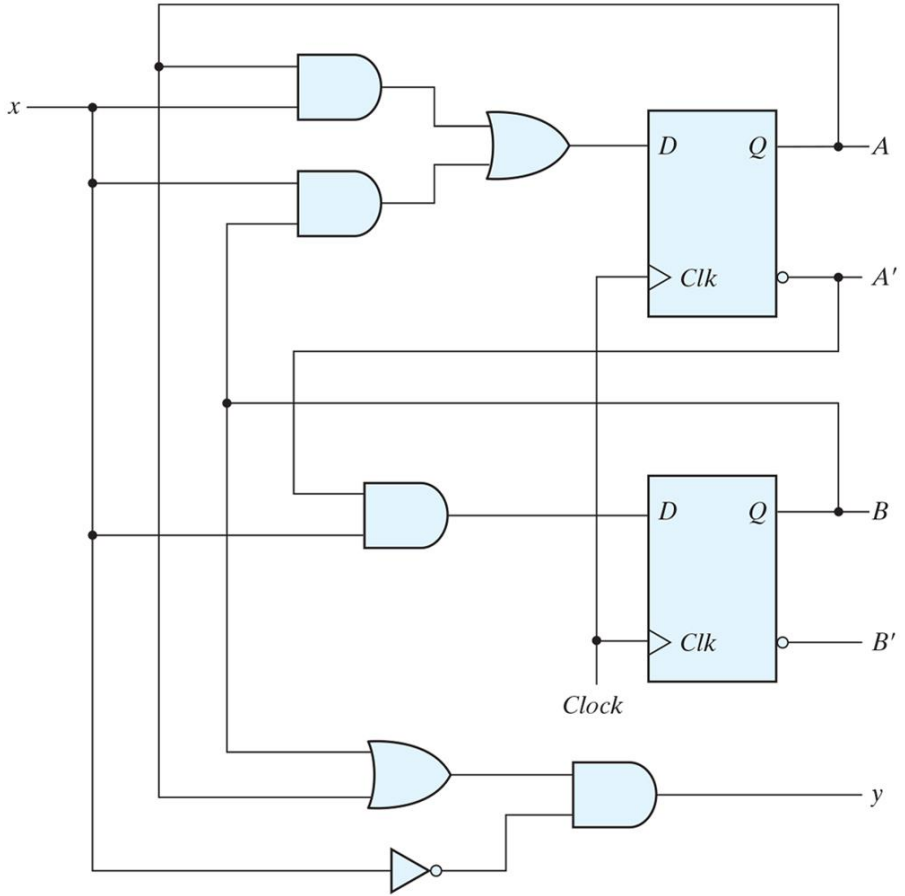
→ Since the D input of a flip-flop determines the value of the next state (i.e., the state reached after the clock transition), it is possible to write a set of state equations directly from the logic diagram as:

- $A(t+1) = A(t)\cdot x(t) + B(t)\cdot x(t)$
- $B(t+1) = A'(t)\cdot x(t)$

→ These equations define a relationship between thw next-states, A(t+1) and B(t+1), and the present-states, A(t) and B(t).

State Equations

→ Example of sequential circuit.



State Equations

→ A state equation is an algebraic expression that specifies the condition for a flip-flop state transition.

→ The left side of the equation, with (t+1), denotes the next state of the flip-flop one clock edge later.

→ The right side of the equation is a Boolean expression that specifies the present state and input conditions that make the next state equal to 1.

→ The Boolean expressions for the state equations can be derived directly from the gates that form the combinational circuit part of the sequential circuit, since the D values of the combinational circuit determine the next state.

→ Similarly, the value of the output can be expressed algebraically as

- $y(t) = (A(t) + B(t))\, x'(t)$

State Table

→ The time sequence of inputs, outputs, and flip-flop states can be enumerated in a state table (sometimes called a transition table).

→ The state table for the circuit of the previous circuit is shown in next table.

→ The table consists of four sections labeled present state, input, next state, and output.

→ The present-state section shows the states of flip-flops A and B at any given time t.

→ The input section gives a value of x for each possible present state.

→ The next-state section shows the states of the flip-flops one clock cycle later, at time t+1.

→ The output section gives the value of y at time t for each present state and input condition.

State Table

→ State Table for the Circuit of the previous circuit.

Present State		Input	Next State		Output
A	*B*	*x*	*A*	*B*	*y*
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

State Table

- → The derivation of a state table requires listing all possible binary combinations of present states and inputs. In this case, we have eight binary combinations from 000 to 111.
- → The next-state values are then determined from the logic diagram or from the state equations.
- → The next state of flip-flop A must satisfy the state equation: $A(t+1) = Ax + Bx$
- → The next-state section in the state table under column A has three 1's where the present state of A and input x are both equal to 1 or the present state of B and input x are both equal to 1.
- → Similarly, the next state of flipflop B is derived from the state equation: $B(t+1) = A'x$
- → The output column is derived from the output equation: $y = Ax' + Bx'$

State Table

- → The state table of any sequential circuit with D-type flip-flops is obtained by the same procedure outlined in the previous example.
- → In general, a sequential circuit with m flip-flops and n inputs needs 2^{m+n} rows in the state table.
- → The binary numbers from 0 through $2^{m+n}-1$ are listed under the present state and input columns.
- → The next-state section has m columns, one for each flip-flop. The binary values for the next state are derived directly from the state equations.
- → The output section has as many columns as there are output variables. Its binary value is derived from the circuit or from the Boolean function in the same manner as in a truth table.
- → It is sometimes convenient to express the state table in a slightly different form having only three sections: present state, next state, and output. The input conditions are enumerated under the next-state and output sections.

State Table

→ Another way to represent the State Table of the previous circuit.

Present State		Next State				Output	
		$x = 0$		$x = 1$		$x = 0$	$x = 1$
A	B	A	B	A	B	y	y
0	0	0	0	0	1	0	0
0	1	0	0	1	1	1	0
1	0	0	0	1	0	1	0
1	1	0	0	1	0	1	0

→ For each present state, there are two possible next states and outputs, depending on the value of the input.

→ One form may be preferable to the other, depending on the application.

State Diagram

→ The information available in a state table can be represented graphically in the form of a state diagram.

→ In this type of diagram, a state is represented by a circle, and the (clock-triggered) transitions between states are indicated by directed lines connecting the circles.

→ Each line originates at a present state and terminates at a next state, depending on the input applied when the circuit is in the present state.

→ The state diagram of the sequential circuit of the previous example is shown in next slide.

→ The state diagram provides the same information as the state table and is obtained directly from state table.

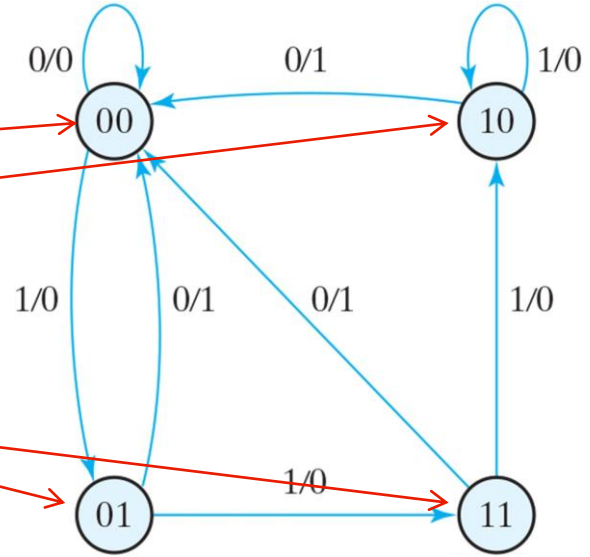
→ The binary number inside each circle identifies the state of the flip-flops.

→ The directed lines are labeled with two binary numbers separated by a slash. The input value during the present state is labeled first, and the number after the slash gives the output during the present state with the given input.

State Diagram

→ State diagram of the circuit of the example.

Present State		Next State				Output	
		$x = 0$		$x = 1$		$x = 0$	$x = 1$
A	B	A	B	A	B	y	y
0	0	0	0	0	1	0	0
0	1	0	0	1	1	1	0
1	0	0	0	1	0	1	0
1	1	0	0	1	0	1	0



State Diagram

- → It is important to remember that the bit value listed for the output along the directed line occurs during the present state and with the indicated input and has nothing to do with the transition to the next state.
- → For example, the directed line from state 00 to 01 is labeled 1/0, meaning that when the sequential circuit is in the present state 00 and the input is 1, the output is 0.
- → After the next clock cycle, the circuit goes to the next state, 01, as determined by the directed edge from 00 to 01.
- → If the input changes to 0, then the output becomes 1, but if the input remains at 1, the output stays at 0.
- → This information is obtained from the state diagram along the two directed lines emanating from the circle with state 01.
- → A directed line connecting a circle with itself indicates that no change of state occurs.

Sequential Circuit Analysis

→ The logic diagram of a sequential circuit consists of flip-flops and gates.

→ The interconnections among the gates form a combinational circuit and may be specified algebraically with Boolean expressions.

→ The knowledge of the type of flip-flops and a list of the Boolean expressions of the combinational circuit provide the information needed to draw the logic diagram of the sequential circuit.

→ The part of the combinational circuit that generates external outputs is described algebraically by a set of Boolean functions called output equations.

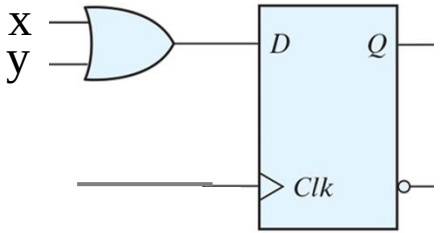
→ The part of the circuit that generates the inputs to flip-flops is described algebraically by a set of Boolean functions called flip-flop input equations (or, sometimes, excitation equations).

Sequential Circuit Analysis

→ We will adopt the convention of using the flip-flop input symbol to denote the input equation variable and a subscript to designate the name of the flipflop output.

→ For example, the following input equation specifies an OR gate with inputs x and y connected to the D input of a flip-flop whose output is labeled with the symbol Q:

$$D_Q = x + y$$

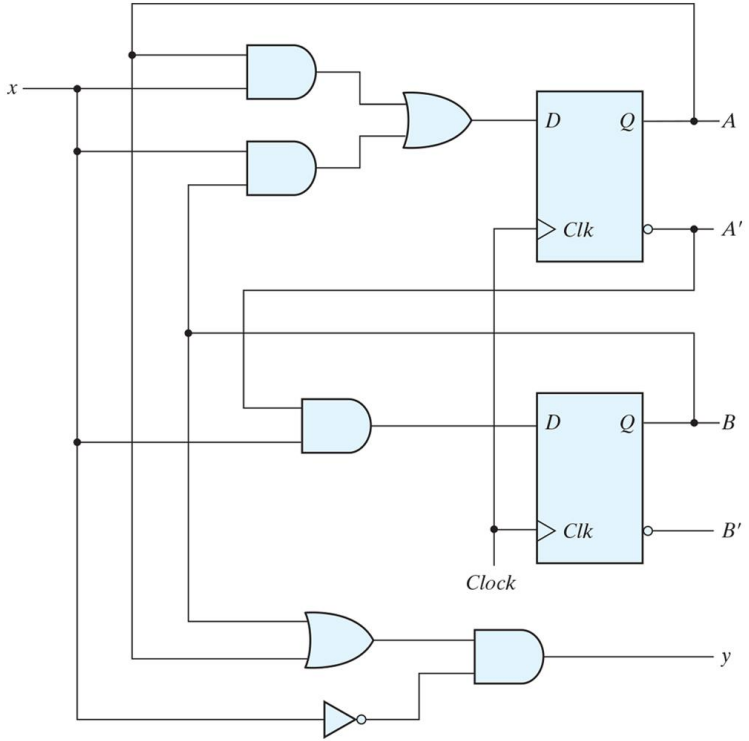


Sequential Circuit Analysis

→ The sequential circuit of the previous example consists of two D flip-flops A and B, an input x, and an output y. The logic diagram of the circuit can be expressed algebraically with two flip-flop input equations and an output equation:

- $D_A = A\,x + B\,x$
- $D_B = A'\,x$
- $y = (A + B)\,x'$

→ The symbol D_A specifies the data input of a D flip-flop labeled A. D_B specifies the data input of a second D flip-flop labeled B.



Sequential Circuit Analysis

→ As an example, lets analyze a sequential circuit with a D flip-flop input as

$$D_A = A \oplus x \oplus y$$

→ The D_A symbol implies a D flip-flop with output A.

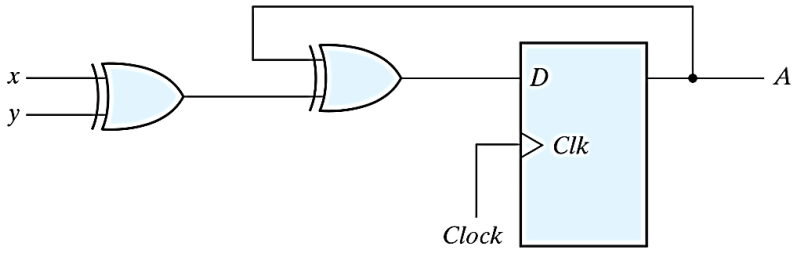
→ The x and y variables are the inputs to the circuit.

→ No output equations are given, which implies that the output comes from the output of the flip-flop.

→ The logic diagram is obtained from the input equation and is drawn in next slide.

Sequential Circuit Analysis

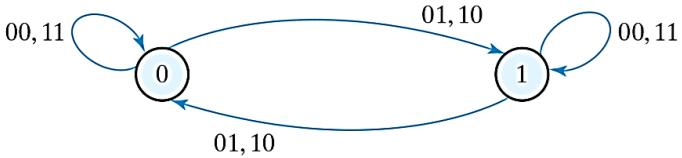
- → Sequential circuit with D flip-flop.
- → The state table has one column for the present state of flip-flop *A*, two columns for the two inputs, and one column for the next state of *A*.
- → The binary numbers under *Axy* are listed from 000 through 111 as shown in figure (b).
- → The next-state values are obtained from the state equation.



(a) Circuit diagram

Present state	Inputs	Next state
A	*x* *y*	*A*
0	0 0	0
0	0 1	1
0	1 0	1
0	1 1	0
1	0 0	1
1	0 1	0
1	1 0	0
1	1 1	1

(b) State table



(c) State diagram

Sequential Circuit Analysis

→ A state table consists of four sections: present state, inputs, next state, and outputs.

→ The first two are obtained by listing all binary combinations.

→ The output section is determined from the output equations.

→ The next-state values are evaluated from the state equations.

→ For a D-type flip-flop, the state equation is the same as the input equation.

→ When a flip-flop other than the D type is used, such as JK or T, it is necessary to refer to the corresponding characteristic table or characteristic equation to obtain the next-state values.

Sequential Circuit Analysis

→ The next-state values of a sequential circuit that uses JK- or T-type flipflops can be derived as follows:

1. Determine the flip-flop input equations in terms of the present state and input variables.
2. List the binary values of each input equation.
3. Use the corresponding flip-flop characteristic table to determine the next-state values in the state table.

Change in state Q(t) → Q(t+1)	D flip-flop	T flip-flop	JK flip-flop
	D	T	JK
0 → 0	0	0	00 or 01 = 0X
0 → 1	1	1	10 or 11 = 1X
1 → 0	0	1	01 or 11 = X1
1 → 1	1	0	00 or 10 = X0

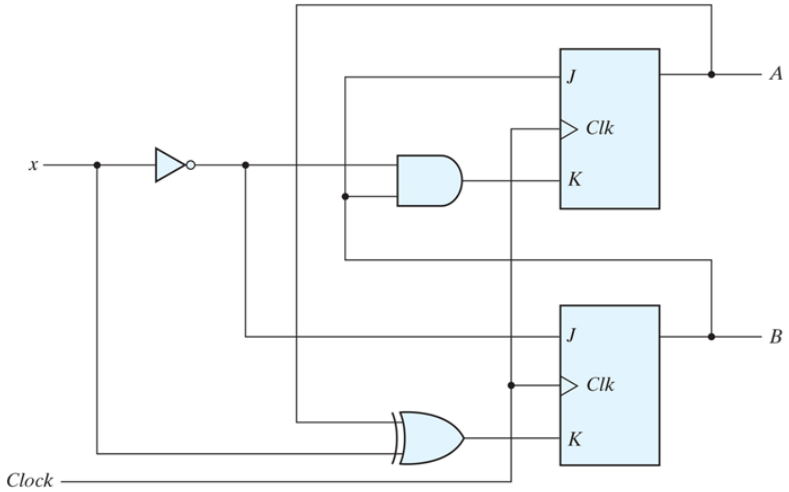
Sequential Circuit Analysis

- As an example, consider the sequential circuit with two JK flip-flops A and B and one input x, as shown in next slide.
- The circuit has no outputs; therefore, the state table does not need an output column.
- The outputs of the flip-flops may be considered as the outputs in this case.
- The circuit can be specified by the flip-flop input equations.
  - $J_A = B \quad K_A = B \cdot x'$
  - $J_B = x' \quad K_B = A \oplus x = A' \cdot x + A \cdot x'$

Sequential Circuit Analysis

→ Sequential circuit with JK flip-flop.

Present State		Input	Next State		Flip-Flop Inputs			
A	B	x	A	B	J_A	K_A	J_B	K_B
0	0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	1
0	1	0	1	1	1	1	1	0
0	1	1	1	0	1	0	0	1
1	0	0	1	1	0	0	1	1
1	0	1	1	0	0	0	0	0
1	1	0	0	0	1	1	1	1
1	1	1	1	1	1	0	0	0



Sequential Circuit Analysis

→ The binary values listed under the columns labeled flip-flop inputs are not part of the state table, but they are needed for the purpose of evaluating the next state as specified in step 2 of the procedure.

→ These binary values are obtained directly from the four input equations in a manner similar to that for obtaining a truth table from a Boolean expression.

→ The next state of each flip-flop is evaluated from the corresponding J and K inputs and the characteristic table of the JK flip-flop listed in the table. There are four cases to consider:

- When J=1 and K=0, the next state is 1.
- When J=0 and K=1, the next state is 0.
- When J=K=0, there is no change of state, and the next-state value is the same as that of the present state.
- When J=K=1, the next-state bit is the complement of the present-state bit.

Sequential Circuit Analysis

→ Examples of the last two cases occur in the table when the present state AB is 10 and input x is 0. J_A and K_A are both equal to 0 and the present state of A is 1. Therefore, the next state of A remains the same and is equal to 1.

→ In the same row of the table, J_B and K_B are both equal to 1. Since the present state of B is 0, the next state of B is complemented and changes to 1.

→ The next-state values can also be obtained by evaluating the state equations from the characteristic equation.

→ This is done by using the following procedure:

1. Determine the flip-flop input equations in terms of the present state and input variables.
2. Substitute the input equations into the flip-flop characteristic equation to obtain the state equations.
3. Use the corresponding state equations to determine the next-state values in the state table.

Sequential Circuit Analysis

→ In the examples circuit, the characteristic equations for the flip-flops are obtained by substituting A or B for the name of the flip-flop, instead of Q:

$$A(t+1) = JA' + K'A$$

$$B(t+1) = JB' + K'B$$

← *from definition of* JK flip-flop

→ Substituting the values of J_A and K_A from the input equations (as shown in the circuit of slide 27), we obtain the state equation for A

$$A(t+1) = BA' + (Bx')'A = A'B + AB' + Ax$$

→ The state equation provides the bit values for the column headed “Next State” for A in the state table.

Sequential Circuit Analysis

→ Similarly, the state equation for flip-flop B can be derived from the characteristic equation by substituting the values of J_B and K_B :

$$B(t+1) = x' B' + (A \oplus x)' B = B'x' + AB\, x + A'Bx'$$

→ The state equation provides the bit values for the column headed “Next State” for B in the state table.

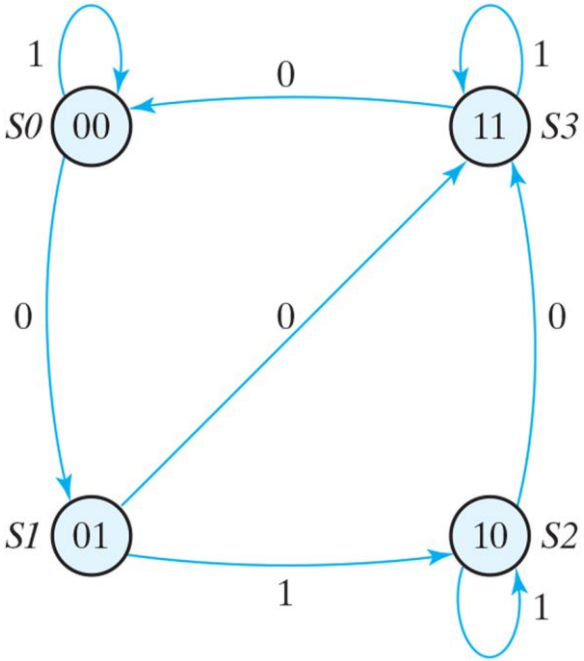
→ Note that the columns in the table headed “Flip-Flop Inputs” are not needed when state equations are used.

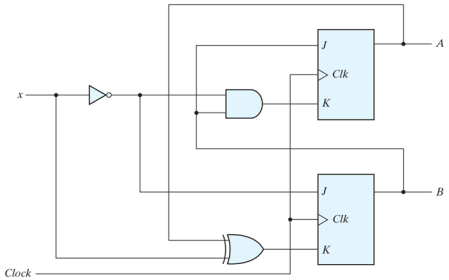
→ The state diagram of the sequential circuit is shown in next slide.

→ Note that since the circuit has no outputs, the directed lines out of the circles are marked with one binary number only, to designate the value of input x.

Sequential Circuit Analysis

→ State diagram of the circuit of the example.





Present State		Input	Next State		Flip-Flop Inputs			
A	B	x	A	B	J_A	K_A	J_B	K_B
0	0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	1
0	1	0	1	1	1	1	1	0
0	1	1	1	0	1	0	0	1
1	0	0	1	1	0	0	1	1
1	0	1	1	0	0	0	0	0
1	1	0	0	0	1	1	1	1
1	1	1	1	1	1	0	0	0

Sequential Circuit Analysis

→ The analysis of a sequential circuit with T flip-flops follows the same procedure outlined for JK flip-flops.

→ The next-state values in the state table can be obtained by using either the characteristic table listed in for T flip-flop or the characteristic equation

$$Q(t+1) = T \oplus Q = T' Q + T Q'$$

→ Now consider, as an example, the sequential circuit shown in next slide. It has two flipflops A and B, one input x, and one output y and can be described algebraically by two input equations and an output equation:

$$T_A = B\,x \quad T_B = x \quad y = AB$$

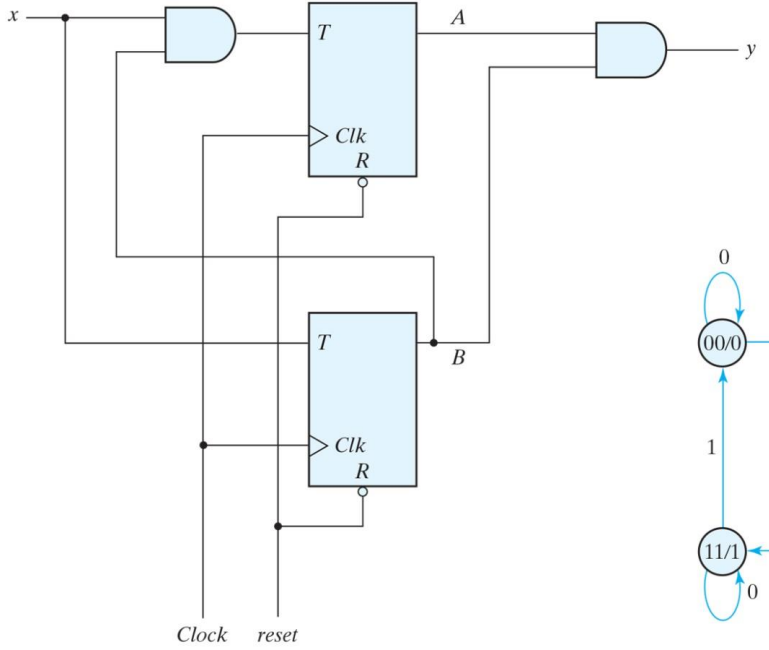
→ The state table for the circuit is listed in the table.

→ The values for y are obtained from the output equation.

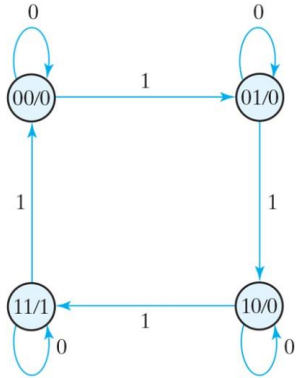
Sequential Circuit Analysis

→ Sequential circuit with T flip-flops (Binary Counter).

Present State		Input	Next State		Output
A	B	x	A	B	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	1



(a) Circuit diagram



(b) State diagram

Sequential Circuit Analysis

→ In the circuit, the values for the next state can be derived from the state equations by substituting T_A and T_B in the characteristic equations, yielding

$$A(t+1) = (Bx)'A + (Bx)A' = A\,B' + A\,x' + A'\,B\,x$$

$$B(t+1) = x \oplus B$$

→ The next-state values for A and B in the state table are obtained from the expressions of the two state equations.

→ As long as input x is equal to 1, the circuit behaves as a binary counter with a sequence of states 00, 01, 10, 11, and back to 00.

→ When x=0, the circuit remains in the same state. Output y is equal to 1 when the present state is 11. Here, the output depends on the present state only and is independent of the input.

→ The two values inside each circle and separated by a slash are for the present state and output.

Mealy and Moore Circuits

→ The most general model of a sequential circuit has inputs, outputs, and internal states.

→ It is customary to distinguish between two models of sequential circuits: the Mealy model and the Moore model. Both are shown in next slide.

→ They differ only in the way the output is generated:

- In the Mealy model, the output is a function of both the present state and the input.
- In the Moore model, the output is a function of only the present state.

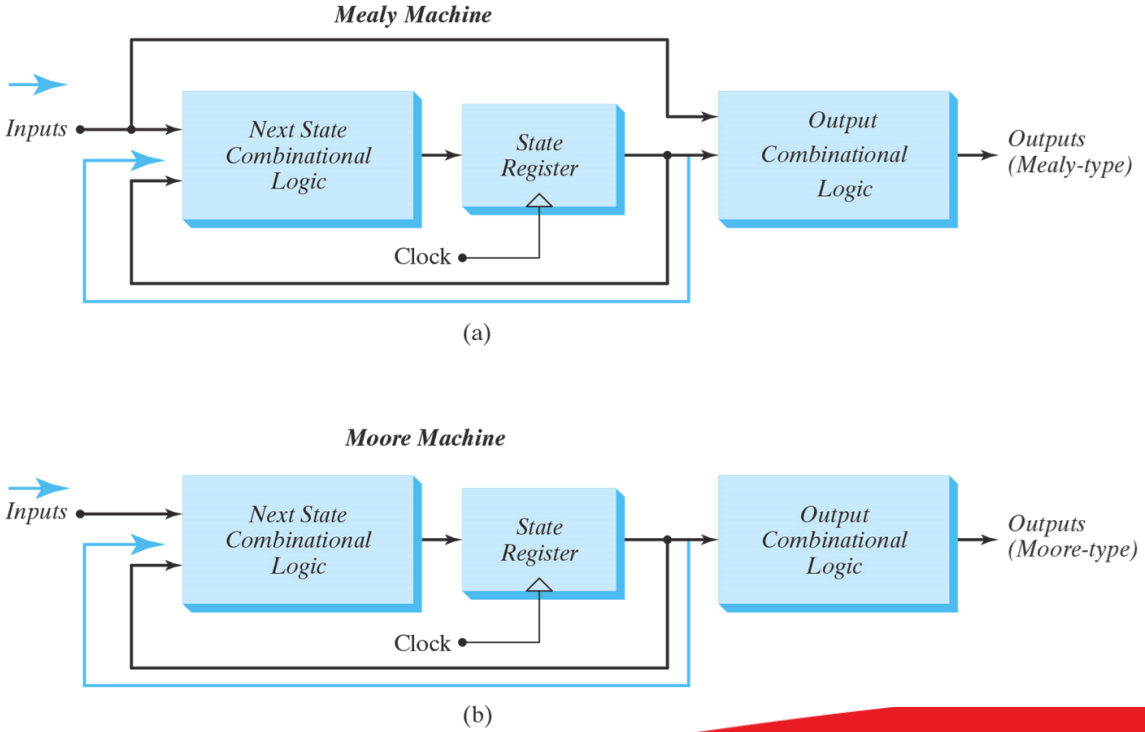
→ A circuit may have both types of outputs.

→ The two models of a sequential circuit are commonly referred to as a finite state machine, abbreviated FSM.

→ The Mealy model of a sequential circuit is referred to as a Mealy FSM or Mealy machine. The Moore model is referred to as a Moore FSM or Moore machine.

Mealy and Moore Circuits

→ Block diagrams of Mealy and Moore state machines.



Mealy and Moore Circuits

→ In a Moore model, the outputs of the sequential circuit are synchronized with the clock, because they depend only on flip-flop outputs, which are synchronized with the clock.

→ In a Mealy model, the outputs may change if the inputs change during the clock cycle. Moreover, the outputs may have momentary false values because of the delay encountered from the time that the inputs change and the time that the flipflop outputs change to their final values.

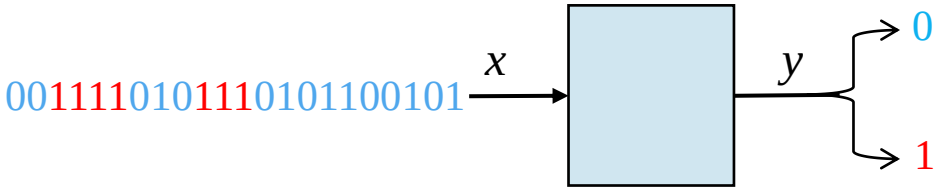
→ In order to synchronize a Mealy-type circuit, the inputs of the sequential circuit must be synchronized with the clock and the outputs must be sampled immediately before the clock edge.

→ The inputs are changed at the inactive edge of the clock to ensure that the inputs to the flip-flops stabilize before the active edge of the clock occurs.

→ Thus, the output of the Mealy machine is the value that is present immediately before the active edge of the clock.

Sequential Circuit Design

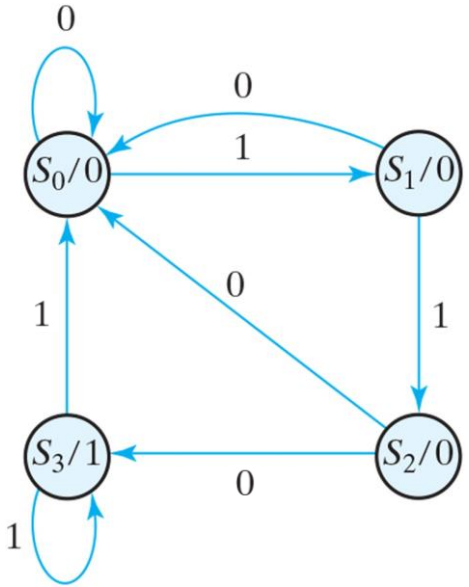
→ **Example 1**: Suppose we wish to design a circuit that detects a sequence of three or more consecutive 1's in a string of bits coming through an input line (i.e., the input is a serial bit stream).



→ Class notes:

Sequential Circuit Design

→ **Example 1**: State diagram, and state table for sequence detector.



Present State		Input	Next State		Output
A	*B*	*x*	*A*	*B*	*y*
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1

Sequential Circuit Design

→ **Example 1**: class notes

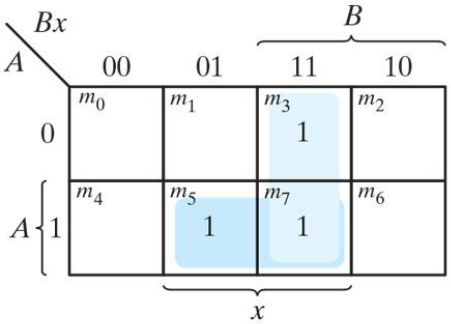
Present State		Input	Next State		Output
A	*B*	*x*	*A*	*B*	*y*
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1

Sequential Circuit Design

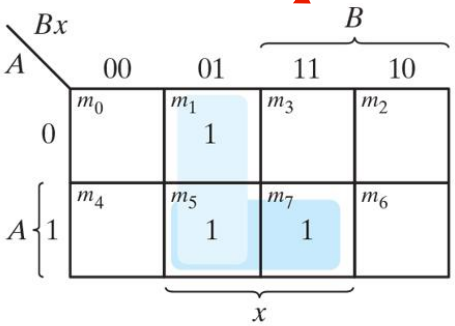
→ **Example 1**: K-Maps for sequence detector.

→ Using D flip-flops

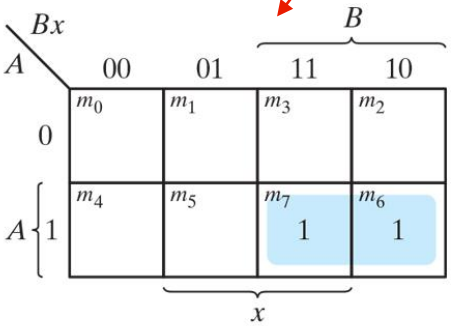
Present State		Input	Next State		Output
A	B	x	A	B	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1



$D_A = Ax + Bx$



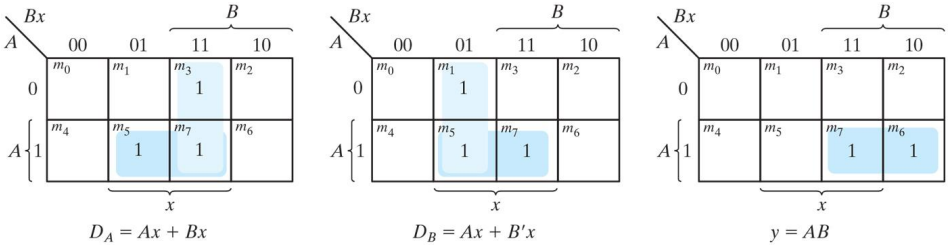
$D_B = Ax + B'x$

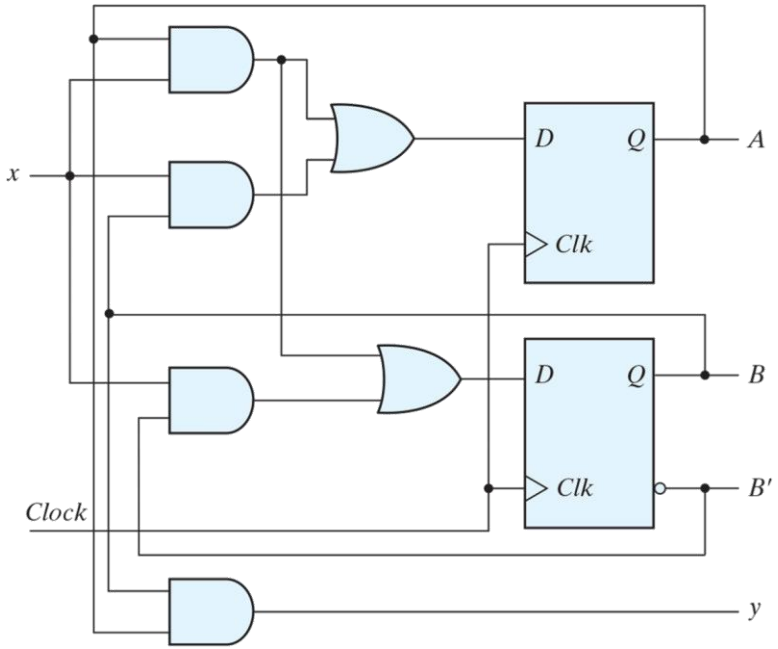


$y = AB$

Sequential Circuit Design

→ **Example 1**: Logic diagram of a Moore-type sequence detector.

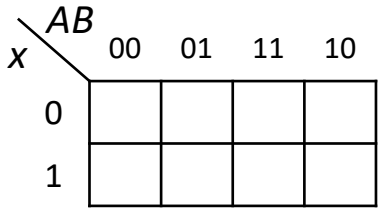




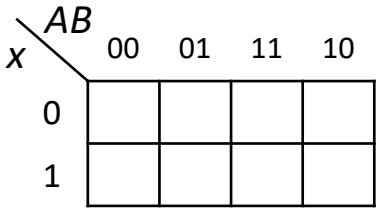
Sequential Circuit Design

→ **Example 1**: redesign using JK flip-flop.

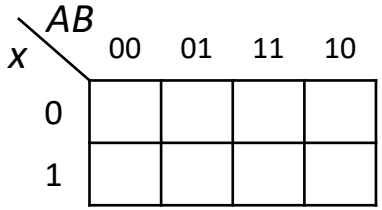
Present State		Input	Next State		Output
A	*B*	*x*	*A*	*B*	*y*
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1



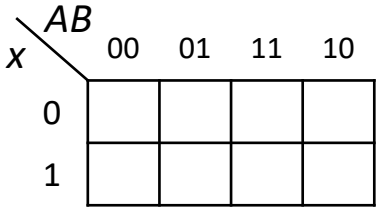
J_A



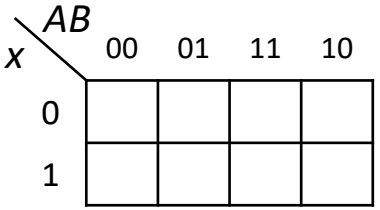
K_A



J_B



K_B



y

Change in state Q(t) → Q(t+1)	JK flip-flop JK
0 → 0	00 or 01 = 0X
0 → 1	10 or 11 = 1X
1 → 0	01 or 11 = X1
1 → 1	00 or 10 = X0

Sequential Circuit Design

→ **Example 1**: redesign using JK flip-flop.

Present State		Input	Next State		Output
A	B	x	A	B	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1

x \ AB	00	01	11	10
0	0	0	x	x
1	0	1	x	x

J_A

x \ AB	00	01	11	10
0	x	x	1	1
1	x	x	0	0

K_A

x \ AB	00	01	11	10
0	0	x	x	0
1	1	x	x	1

J_B

x \ AB	00	01	11	10
0	x	1	1	x
1	x	1	0	x

K_B

x \ AB	00	01	11	10
0	0	0	1	0
1	0	0	1	0

y

Change in state Q(t) → Q(t+1)	JK flip-flop JK
0 → 0	00 or 01 = 0X
0 → 1	10 or 11 = 1X
1 → 0	01 or 11 = X1
1 → 1	00 or 10 = X0

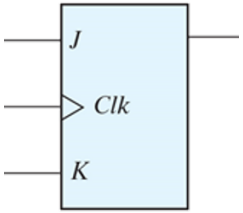
Sequential Circuit Design

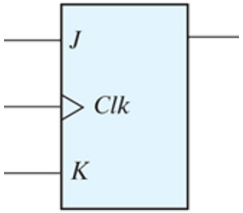
→ **Example 1**: redesign using JK flip-flop.

$J_A = x \cdot B$ $K_A = x'$

$J_B = x$ $K_B = x' + A'$

$y = A \cdot B$





x \ AB	00	01	11	10
0	0	0	x	x
1	0	1	x	x

J_A

x \ AB	00	01	11	10
0	x	x	1	1
1	x	x	0	0

K_A

x \ AB	00	01	11	10
0	0	x	x	0
1	1	x	x	1

J_B

x \ AB	00	01	11	10
0	x	1	1	x
1	x	1	0	x

K_B

x \ AB	00	01	11	10
0	0	0	1	0
1	0	0	1	0

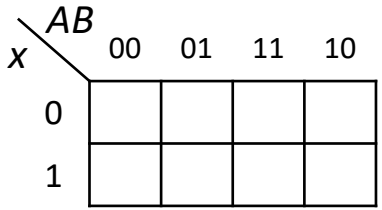
y

← Mealy or Moore?

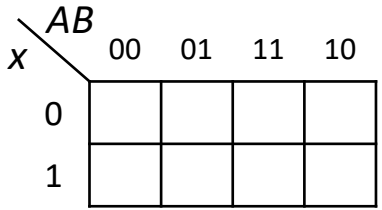
Sequential Circuit Design

→ **Example 1**: redesign using T flip-flop.

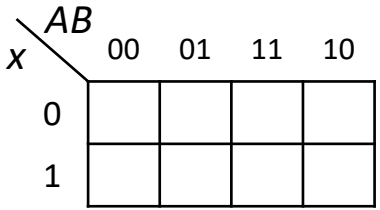
Present State		Input	Next State		Output
A	*B*	*x*	*A*	*B*	*y*
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1



T_A



T_B



y

Change in state Q(t) → Q(t+1)	T flip-flop
	T
0 → 0	0
0 → 1	1
1 → 0	1
1 → 1	0

Sequential Circuit Design

→ **Example 1**: redesign using T flip-flop.

Present State		Input	Next State		Output
A	*B*	*x*	*A*	*B*	*y*
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1

Change in state Q(t) → Q(t+1)	T flip-flop
	T
0 → 0	0
0 → 1	1
1 → 0	1
1 → 1	0

x \ AB	00	01	11	10
0	0	0	1	1
1	0	1	0	0

T_A

x \ AB	00	01	11	10
0	0	1	1	0
1	1	1	0	1

T_B

x \ AB	00	01	11	10
0	0	0	1	0
1	0	0	1	0

y

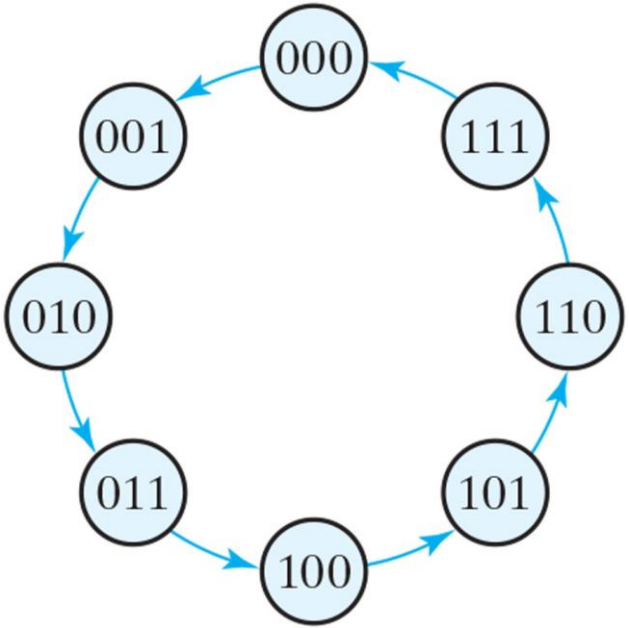
Sequential Circuit Design

→ **Example 2**: three-bit binary counter.

→ State diagram of three-bit binary counter.

→ State Table for Three-Bit Counter.

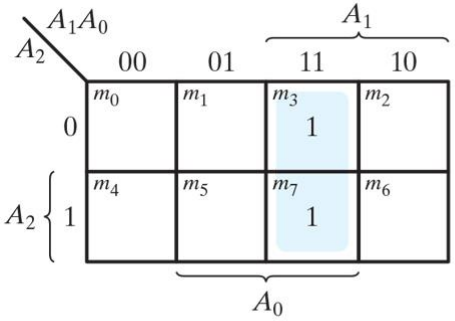
Present State			Next State			Flip-Flop Inputs		
A_2	A_1	A_0	A_2	A_1	A_0	T_{A2}	T_{A1}	T_{A0}
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	1	1	1	0	0	1
1	1	1	0	0	0	1	1	1



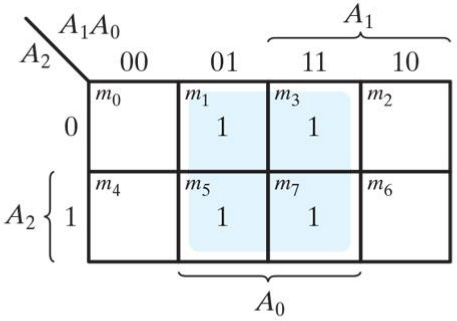
Sequential Circuit Design

→ **Example 2**: Maps for three-bit binary counter.

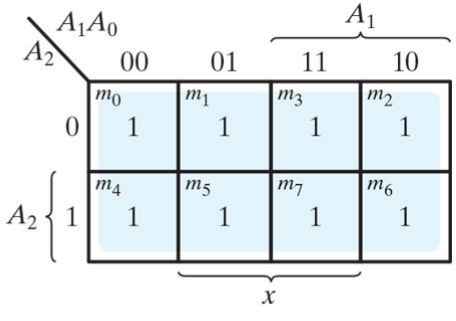
Present State			Next State			Flip-Flop Inputs		
A_2	A_1	A_0	A_2	A_1	A_0	T_{A2}	T_{A1}	T_{A0}
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	1	1	1	0	0	1
1	1	1	0	0	0	1	1	1



$T_{A2} = A_1A_0$



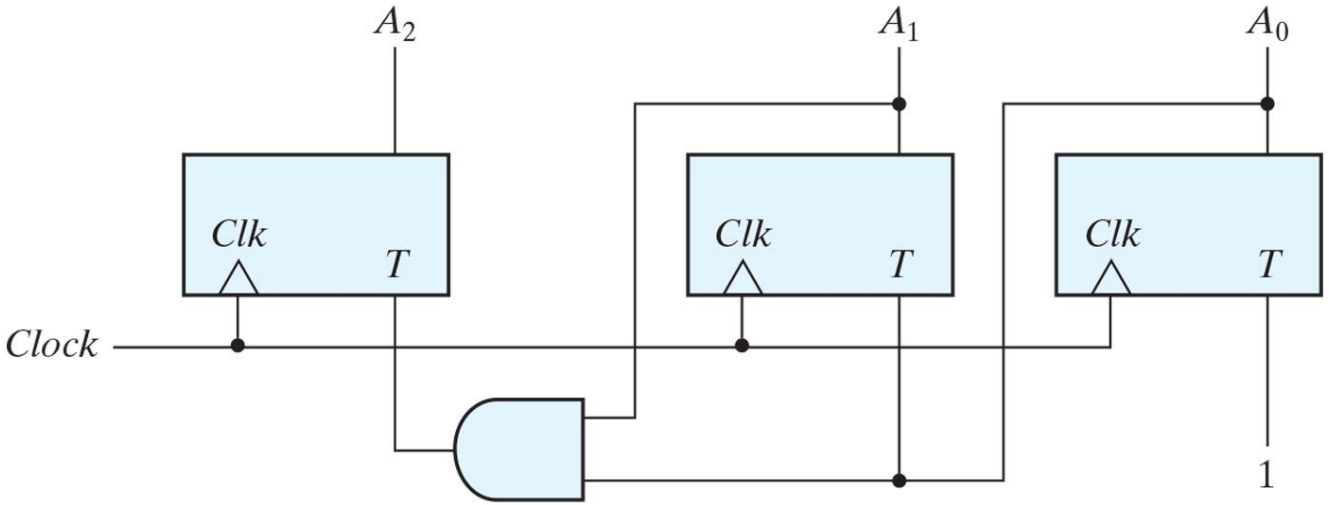
$T_{A1} = A_0$



$T_{A0} = 1$

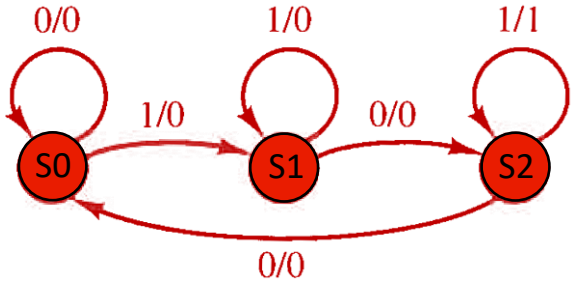
Sequential Circuit Design

→ **Example 2**: Logic diagram of three-bit binary counter.



Sequential Circuit Design

→ **Example 3**: Consider the state diagram.



→ How many flip flops required to design a circuit for this state diagram?

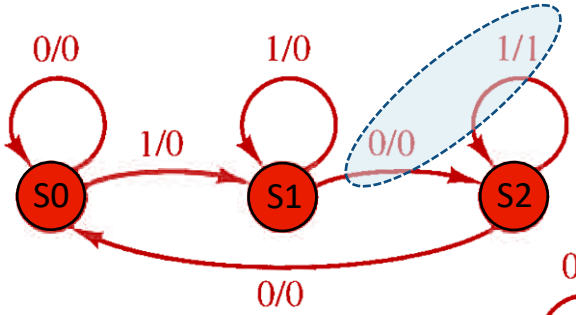
→ This state diagram represents which one, a Mealy or a Moore machine?

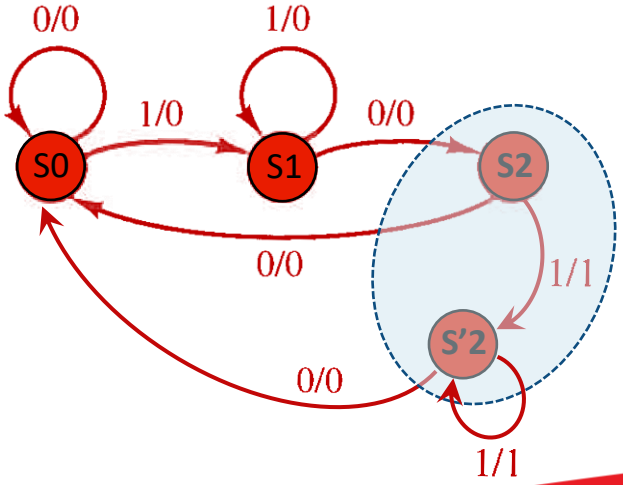
→ Design a circuit for this diagram using logic gates and

a) D flip-flops
b) T flip-flops
c) JK flip-flops

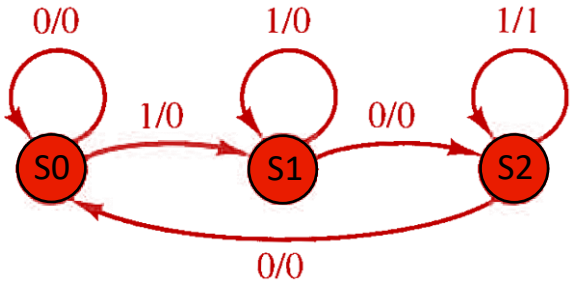
Sequential Circuit Design: Example 3

1. Mealy or Moore

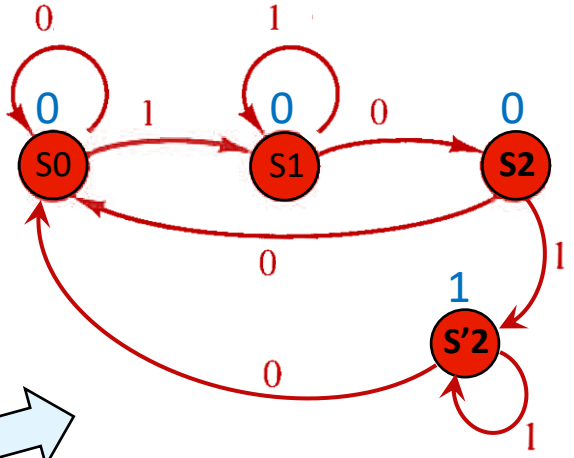




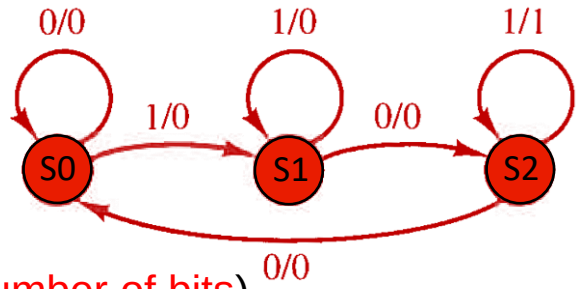
Mealy



Moore



Sequential Circuit Design: Example 3



1. Binary coding of the states:

→ We can code the states in many (infinite) different ways:

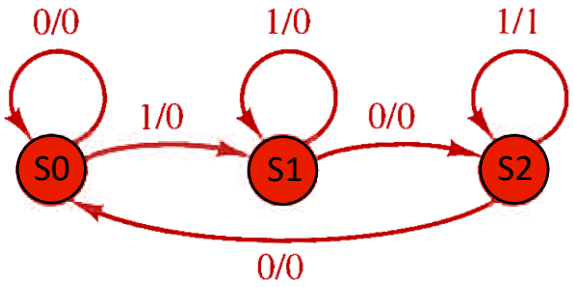
- → Two-bit coded (in this example it is the minimum required number of bits)
  - → $S0 \rightarrow 00$, $S1 \rightarrow 01$, $S2 \rightarrow 10$
  - → $S0 \rightarrow 00$, $S1 \rightarrow 01$, $S2 \rightarrow 11$
  - → $S0 \rightarrow 00$, $S1 \rightarrow 11$, $S2 \rightarrow 10$
  - → … … …

Question: How many two-bit code assignments are possible?

- → Three-bit coded (in this example it is a code with redundancy)
  - → $S0 \rightarrow 000$, $S1 \rightarrow 001$, $S2 \rightarrow 010$
  - → $S0 \rightarrow 000$, $S1 \rightarrow 101$, $S2 \rightarrow 011$
  - → … … …
- → And many others!

Question: Which code is better?
Question: What are the cost and befits of each coding?
Question: Can you design codes with only one bit difference between neighbor states?

Sequential Circuit Design: Example 3



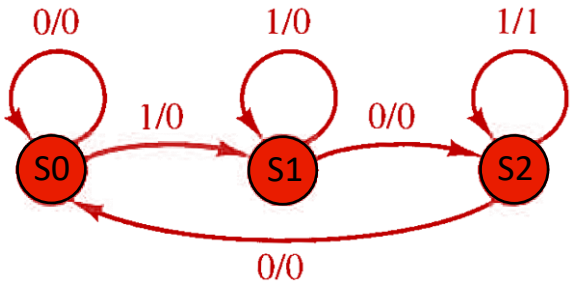
2. State (transition) Table of the diagram,
   → Let's use a two-bit code:
   → Current state: $S_i(t) = A(t)B(t)$, Next state $S_i(t+1) = A(t+1)B(t+1)$

Current State	input	Next State	Output
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S2	1

S0 → 00,
S1 → 01,
S2 → 10

A(t)	B(t)	x	A(t+1)	B(t+1)	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	0	1	0
1	0	0	0	0	0
1	0	1	1	0	1
1	1	0	x	x	x
1	1	1	x	x	x

Sequential Circuit Design: Example 3



3. Choosing flip-flops type and designing their input circuits.
   a) Let's use D flip-flops and call their inputs as D_A and D_B

A(t)	B(t)	x	A(t+1)	B(t+1)	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	0	1	0
1	0	0	0	0	0
1	0	1	1	0	1
1	1	0	x	x	x
1	1	1	x	x	x

A(t+1)

x \ AB	00	01	11	10
0	0	1	x	0
1	0	0	x	1

D_A

B(t+1)

x \ AB	00	01	11	10
0	0	0	x	0
1	1	1	x	0

D_B

y

x \ AB	00	01	11	10
0	0	0	x	0
1	0	0	x	1

Sequential Circuit Design: Example 3

→ Circuit diagram with D flip-flops (check the class notes for circuit)

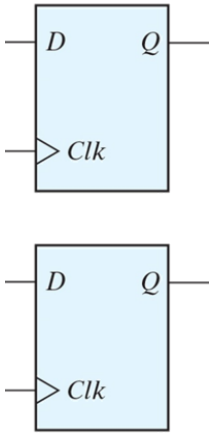
x \ AB	00	01	11	10
0	0	1	x	0
1	0	0	x	1

D_A

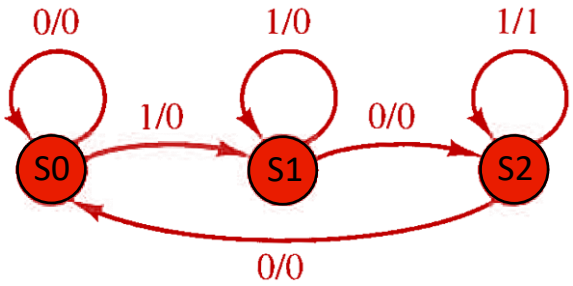
x \ AB	00	01	11	10
0	0	0	x	0
1	1	1	x	0

D_B

x \ AB	00	01	11	10
0	0	0	x	0
1	0	0	x	1



Class notes: Example 3



3. Choosing flip-flops type and designing their input circuits.
   a) Let's use T flip-flops and call their inputs as T_A and T_B

A(t)	B(t)	x	A(t+1)	B(t+1)	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	0	1	0
1	0	0	0	0	0
1	0	1	1	0	1
1	1	0	x	x	x
1	1	1	x	x	x

A(t+1)

x \ AB	00	01	11	10
0	0	1	x	1
1	0	0	x	0

T_A

B(t+1)

x \ AB	00	01	11	10
0	0	1	x	0
1	1	0	x	0

T_B

y

x \ AB	00	01	11	10
0	0	0	x	0
1	0	0	x	1

Sequential Circuit Design: Example 3

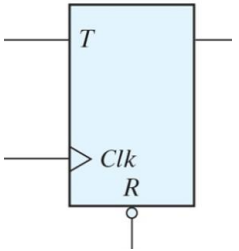
→ Circuit diagram with T flip-flops (check the class notes for circuit)

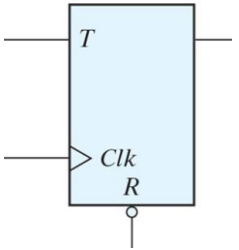
x \ AB	00	01	11	10
0	0	1	x	1
1	0	0	x	0

T_A

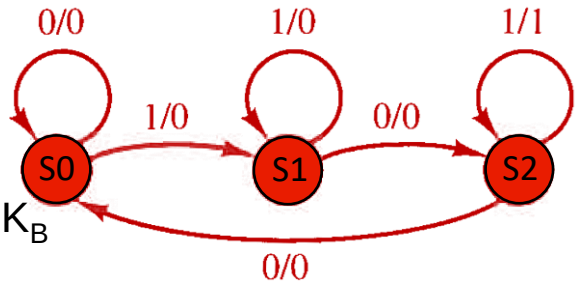
x \ AB	00	01	11	10
0	0	1	x	0
1	1	0	x	0

T_B





Class notes: Example 3



3. Choosing flip-flops type and designing their input circuits.
   a) Let's use JK flip-flops and call their inputs as J_A, K_A and J_B, K_B

A(t)	B(t)	x	A(t+1)	B(t+1)	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	0	1	0
1	0	0	0	0	0
1	0	1	1	0	1
1	1	0	x	x	x
1	1	1	x	x	x

A(t+1)

x \ AB	00	01	11	10
0	0	1	x	x
1	0	0	x	x

J_A

x \ AB	00	01	11	10
0	x	x	x	1
1	x	x	x	0

K_A

B(t+1)

x \ AB	00	01	11	10
0	0	x	x	0
1	1	x	x	0

J_B

x \ AB	00	01	11	10
0	x	1	x	x
1	x	0	x	1

K_B

Reminder

Q(t) → Q(t+1)	JK
0 → 0	0X
0 → 1	1X
1 → 0	X1
1 → 1	X0

Sequential Circuit Design: Example 3

→ Circuit diagram with JK flip-flops (check the class notes for circuit)

x \ AB	00	01	11	10
0	0	1	x	x
1	0	0	x	x

J_A

x \ AB	00	01	11	10
0	x	x	x	1
1	x	x	x	0

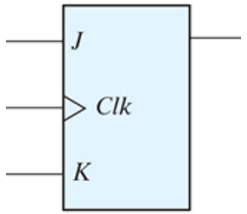
K_A

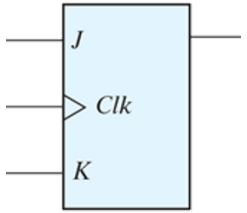
x \ AB	00	01	11	10
0	0	x	x	0
1	1	x	x	0

J_B

x \ AB	00	01	11	10
0	x	1	x	x
1	x	0	x	1

K_B





State Reduction and Assignment

→ Analysis of sequential circuits starts from a circuit diagram and culminates in a state table or diagram.

→ Design (synthesis) of a sequential circuit starts from a set of specifications and culminates in a logic diagram.

→ As we saw before, two sequential circuits may exhibit the same input–output behavior but have a different number of internal states in their state diagram.

→ There are methods to simplify a sequential circuit design by reducing the number of gates and flip-flops it uses.

→ In general, reducing the number of flip-flops reduces the cost of a circuit.

→ The reduction in the number of flip-flops in a sequential circuit is referred to as the state-reduction problem.

State Reduction and Assignment

→ State-reduction algorithms are concerned with procedures for reducing the number of states in a state table, while keeping the external input–output requirements unchanged.

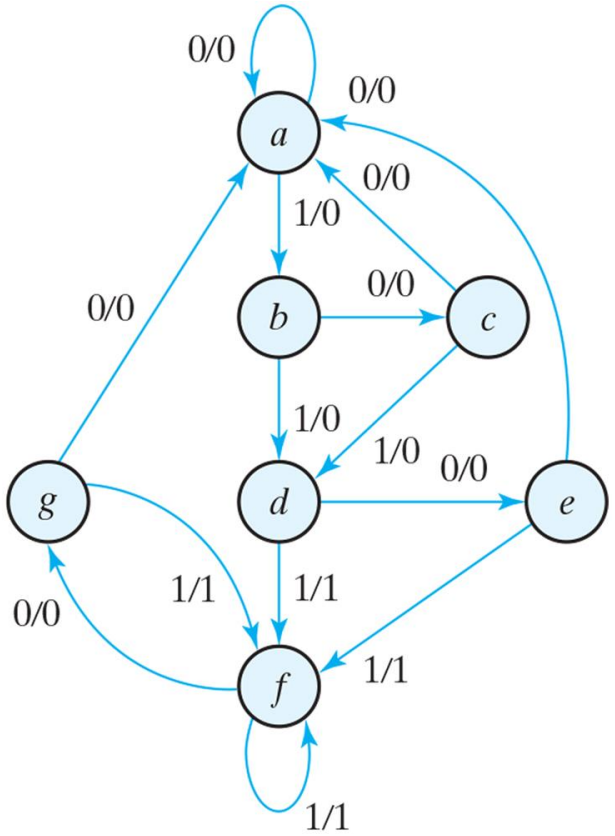
→ Since m flip-flops produce 2^m states, a reduction in the number of states may (or may not) result in a reduction in the number of flip-flops. (why?)

→ An unpredictable effect in reducing the number of flip-flops is that sometimes the equivalent circuit (with fewer flip-flops) may require more combinational gates to realize its next state and output logic!

State Reduction and Assignment

→ Consider a sequential circuit with this state diagram.

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	g	f	0	1
g	a	f	0	1



State Reduction and Assignment

- First, we need the state table; it is more convenient to apply procedures for state reduction with the use of a table rather than a diagram.
- "Two states are equivalent if, for each member of the set of inputs, they give exactly the same output and send the circuit either to the same state or to an equivalent state."
- When two states are equivalent, one of them can be removed without altering the input–output relationships.
- Going through the state table, we look for two present states that go to the same next state and have the same output for both input combinations:
  - States e and g are two such states: They both go to states a and f and have outputs of 0 and 1 for x=0 and x=1, respectively.
  - Therefore, states g and e are equivalent, and one of these states can be removed.

State Reduction and Assignment

→ The procedure of removing a state and replacing it by its equivalent is demonstrated in the table. The row with present state g is removed, and state g is replaced by state e each time it occurs in the columns headed “Next State.”

→ Present state f now has next states e and f and outputs 0 and 1 for x=0 and x=1, respectively.

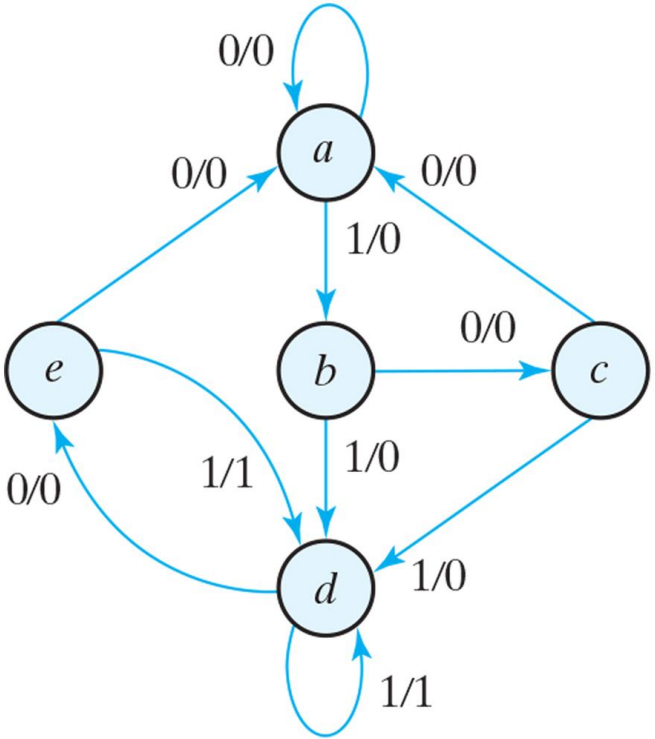
→ The same next states and outputs appear in the row with present state d. Therefore, states f and d are equivalent, and state f can be removed and replaced by d.

→ The final reduced table is shown in next slide.

State Reduction and Assignment

→ Reducing the state table and its reduced state diagram. .

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	e	f	0	1



State Reduction and Assignment

- → Checking each pair of states for equivalency can be done systematically by means of a procedure that employs an implication table, which consists of squares, one for every suspected pair of possible equivalent states.
- → By judicious use of the table, it is possible to determine all pairs of equivalent states in a state table.
- → The sequential circuit of this example was reduced from seven to five states.
- → In general, reducing the number of states in a state table may result in a circuit with less physical hardware. However, the fact that a state table has been reduced to fewer states does not guarantee a saving in the number of flip-flops or the number of gates.
- → In actual practice designers may skip this step because target devices are rich in resources.

State Assignment

- → In order to design a sequential circuit with physical components, it is necessary to assign unique coded binary values to the states.
- → For a circuit with m states, the codes must contain n bits, where $2^n \geq m$.
- → For example, with three bits, it is possible to assign codes to eight states, denoted by binary numbers 000 through 111.
- → In some circuits number of required binary assignments are less than available numbers, and we are left with unused codes.
- → Unused state codes are treated as don't-care conditions during the design.
- → Since don't-care conditions usually help in obtaining a simpler circuit, it is more likely but not certain that the circuit with five states will require fewer combinational gates than the one with seven states.

State Assignment

- → The simplest way to code states is to use the integers in binary counting order (0 to m).
- → Another similar assignment is the Gray code, where only one bit in the code group changes when going from one number to the next. This code makes it easier for the Boolean functions to be placed in a Karnaugh map for simplification.
- → Another possible assignment often used in the design of state machines to control datapath units is the one-hot assignment. This configuration uses as many bits as there are states in the circuit. At any given time, only one bit is equal to 1 while all others are kept at 0.
  - → This type of assignment uses one flip-flop per state, which is not an issue for register-rich field-programmable gate arrays.
  - → One-hot encoding usually leads to simpler decoding logic for the next state and output.
  - → One-hot machines can be faster than machines with sequential binary encoding, and the silicon area required by the extra flip-flops can be offset by the area saved by using simpler decoding logic. This trade-off is not guaranteed, so it must be evaluated for a given design.

State Assignment

→ Three Possible Binary State Assignments.

State	Assignment 1, Binary	Assignment 2, Gray Code	Assignment 3, One-Hot
a	000	000	00001
b	001	001	00010
c	010	011	00100
d	011	010	01000
e	100	110	10000

Design Procedure: conclusion

1. From the word description and specifications of the desired operation, derive a state diagram for the circuit.
2. Reduce the number of states if necessary.
3. Assign binary values to the states.
4. Obtain the binary-coded state table.
5. Choose the type of flip-flops to be used.
6. Derive the simplified flip-flop input equations and output equations.
7. Draw the logic diagram.